

JPMC KYC Risk Pre-Screening (UBO GraphRAG)

— Solution Summary

Client & Context

- **Client:** JPMorgan Chase (JPMC) — inferred from repo path JPMC_KYC, README deployment paths (JPMC_UBO, JPMC_UBO_local), and Azure VM layout under AICoE_Solutions/JPMC_UBO
- **Vertical:** Finance (KYC / AML / UBO corporate onboarding)
- **Project Type:** Agentic System + Dashboard — React analyst workspace (kyc-ui) backed by a multi-step AI pipeline (ubo-backend) for document ingestion, entity extraction, risk scoring, and analyst review

Business Problem

The client was manually reviewing corporate KYC documents (trust deeds, incorporation records, ownership charts) to extract entities, ownership relationships, and Ultimate Beneficial Owners (UBOs), then scoring geographic, industry, and sanctions exposure before onboarding. That process caused slow onboarding queues, inconsistent risk scoring, missed mandatory policy fields (e.g., offshore_exposure_declaration, ubo_identification), and repetitive analyst work drafting follow-up emails when submissions were incomplete.

Solution Overview

Affine Analytics built a **KYC Risk Pre-Screening** platform branded in the UI as “Affine Analytics / KYC Risk Pre-Screening.” Analysts upload .pdf, .txt, or .docx files per project via the Document Upload page; the backend stores files in Azure Blob Storage, uses Azure OpenAI to extract structured entities/relationships/UBOs, validates completeness against kyc_ingestion_policy.json, and either blocks analysis or persists rows to Azure SQL and computes an AI risk score. Results surface in a dashboard (pipeline metrics, risk distribution, missing-info projects), a prioritized Client Queue, a Risk Assessment workspace with radial ownership visualization and downloadable KYC_Risk_Report_{project}.docx, and a Geographic Exposure map with FATF ratings. GraphRAG indexes project documents in the background for per-project Q&A via /chat.

Key Features

- **Dashboard Overview** — shows Total Clients in Pipeline, High-Risk Clients, Projects Missing Information, status breakdown (In-progress / Completed / Verification), risk distribution donut, and daily onboarding volume trend
- **Document Upload** — upload PDF/TXT/DOCX with document type (Trust Deed, Incorporation Record, Ownership Chart, Other) and project assignment; tracks status (Processing, Completed, Insufficient Information, Failed)
- **Missing Details workspace** — lists projects blocked by missing KYC fields with expandable detail modal showing missing_items, missing_item_explanations, and extracted_policy_fields
- **Client Queue** — prioritized table by risk_score with columns Client Name, Entity Type, Country, Industry, Risk Score, Risk Category, Ownership

Complexity, Assigned To, Status, Days in Queue; supports search, status filter, bulk approve, CSV export, analyst assignment

- **Risk Assessment** — per-project score gauge (final_ai_risk_score_0_100), tier badge (Low/Medium/High), five dimension cards (Geographic Exposure, Company Structure, Ownership & UBO Analysis, Industry Classification, Adverse Media & Sanctions), expandable LLM reasoning, radial ownership tree visualization
- **Analyst decision workflow** — Approve, Request More Information (opens LLM-drafted email modal), Escalate, Reject; persists risk_decision_status.json per project
- **Download Full Report** — generates/downloads Word doc via /risk-assessment/report (KYC_Risk_Report_{project}.docx)
- **Request More Information email** — LLM drafts subject/body; analyst edits and sends via SMTP
- **Geographic Exposure** — country table with Region, FATF Risk Rating, # Clients, Sanctions flag, Avg Risk Score; region donut and top-15 bar chart
- **Analytics & Reports** — portfolio KPI cards and chart templates (Risk Category Trend, Processing Time Distribution, AI Model Accuracy radar, Top Risk Factors); exportable report catalog (Monthly Risk Summary, High-Risk Client Register, etc.)
- **Settings & Thresholds** — UI-only risk weight sliders (companyStructure 20%, geographicExposure 24%, industryClassification 16%, ownershipUbo 24%, adverseMediaSanctions 16%), Low/Medium/High threshold sliders, auto-approval rules, notification toggles, local cache clear
- **GraphRAG chat** — per-project document Q&A via POST / chat with method: local|global|drift|basic
- **Login gate** — client-side auth (admin@affine.ai) with Affine branding

AI Agents & Components

Entity Extraction Agent

- **Function:** Extracts structured entities, ownership_relationships, and ubo_registry from uploaded corporate documents
- **Input:** Document text from PDF (PyMuPDF), DOCX (python-docx), or TXT; filename
- **Output:** JSON arrays with fields like entity_name, entity_type, jurisdiction, ownership_percentage, full_name, nationality, passport_number, linked_entities
- **Model/Service:** Azure OpenAI deployment GPT4_LLM_MODEL_DEPLOYMENT_NAME (configured as gpt-5.1); endpoint https://polycode-solutions.openai.azure.com/
- **Position in flow:** Ingestion / processing — first LLM step after blob upload

KYC Policy Completeness Validator

- **Function:** Validates extracted data against kyc_ingestion_policy.json required fields; determines if analysis is blocking
- **Input:** Extracted JSON, source document text, KYC policy JSON (25+ required fields across general_client_information,

- geographic_exposure_data, industry_information, ownership_structure_data, sanctions_background_check_data)
- **Output:** { is_blocking, missing_items, reason, extracted_policy_fields }; sets document status to Insufficient Information when blocking
- **Model/Service:** Azure OpenAI (gpt-5.1), JSON response format, temperature 0
- **Position in flow:** Validation — gates risk scoring pipeline

Missing Information Email Drafter

- **Function:** Drafts client-facing follow-up emails listing missing KYC items with professional tone
- **Input:** client_name, source_doc, missing_items, reason, missing_item_explanations
- **Output:** { subject, body } — subject default “Request for Additional Information – KYC Process”; body starts “Hi {Client Name},” and ends “Regards, Compliance Team”
- **Model/Service:** Azure OpenAI (gpt-5.1), JSON response format
- **Position in flow:** Output / human-in-the-loop — triggered from Document Upload notify or Risk Assessment “Request More Information”

Main Entity Inference Agent

- **Function:** Infers the primary corporate entity a project document describes for ownership visualization center node
- **Input:** Latest document bytes, candidate entity names from SQL
- **Output:** { entity_id, entity_name, reason } via GET /graph/main-entity
- **Model/Service:** Azure OpenAI (gpt-5.1) via call_llm_for_main_entity
- **Position in flow:** Processing — supports radial ownership graph UI

ML Risk Classifier (Rule Stub)

- **Function:** Classifies risk tier from engineered features before weighted scoring
- **Input:** Features: primary_country, fatf_country_risk_band, industry_risk_band, ownership_layers, ownership_opacity, ubo_max_ownership_pct, pep_exposure, sanctions_hits, high_risk_geo_pct, circular_ownership
- **Output:** { risk_tier: Low|Medium|High, confidence: 0.74–0.9 }
- **Model/Service:** Custom rule-based stub in ml_stub.py (not a trained ML model)
- **Position in flow:** Processing — prior for section score calibration

Risk Scoring Agent

- **Function:** Combines geography (0–30), ownership (0–30), regulatory/sanctions (0–20), industry (0–20) into final score; applies hard overrides (FATF blacklist, sanctioned country, circular ownership, PEP exposure, active investigation)
- **Input:** RiskScoringInput with normalized section scores and OverrideFlags
- **Output:** RiskScoringOutput: final_score (0–100), risk_tier, override_triggered, override_reason, LLM reasoning
- **Model/Service:** Deterministic scoring + Azure OpenAI reasoning (AzureOpenAIReasoningLLM)

- **Position in flow:** Processing — core risk engine

Risk Reasoning Generator

- **Function:** Produces UI and report narrative for final summary plus Geography, Ownership, Industry, Sanctions sections
- **Input:** Project context, section scores (/100), final score/tier, feature evidence
- **Output:** final_reasoning (UI paragraph) and report_reasoning bundle persisted to risk_assessment_summary.json
- **Model/Service:** Azure OpenAI (gpt-5.1), parallel asyncio.gather of 6 prompts
- **Position in flow:** Output — feeds Risk Assessment UI and DOCX report generator

UBO Analysis Engine

- **Function:** Computes UBO-specific score from concentration, transparency, complexity, and risk flags
- **Input:** Entity rows, UBO rows, ownership relationships from Azure SQL
- **Output:** ubo_analysis with score_0_100, tier, summary, metrics (total_ubos, max_ownership_pct, pep_count, circular_ownership, etc.), top_ubos
- **Model/Service:** Deterministic Python logic in pipeline_full.py
- **Position in flow:** Processing — supplementary to main risk score

GraphRAG Index & Query Pipeline

- **Function:** Converts documents to TXT, indexes per-project knowledge graph (entities, communities, embeddings), answers analyst questions
- **Input:** Project documents in projects/{project_id}/input/; query string + method (local, global, drift, basic)
- **Output:** Parquet/LanceDB artifacts; natural-language answers via graphrag query
- **Model/Service:** Microsoft GraphRAG library; Azure OpenAI gpt-5.2 (GraphRAG settings) + text-embedding-3-small (1536-dim); LanceDB vector store
- **Position in flow:** Processing / retrieval — background thread after upload; Q&A on demand

KYC Risk Report Generator

- **Function:** Builds formatted Word report with KPI summary, section scores, UBO table, override status, AI reasoning
- **Input:** Risk assessment payload + SQL entity/UBO/relationship rows
- **Output:** .docx bytes stored at {project}/kyc_results/latest/KYC_Risk_Report_{project}.docx
- **Model/Service:** python-docx (no LLM at generation time; uses pre-computed LLM reasoning)
- **Position in flow:** Output

Tech Stack

- **Frontend:** React 19, TypeScript, Vite 8, React Router 7, Tailwind CSS 4, Recharts 3, D3 (zoom/drag/selection), Axios, Zod

- **Backend:** Python 3.13, FastAPI, Uvicorn, Pydantic, Pandas, asyncio threading
- **AI/ML:** Azure OpenAI (gpt-5.1 extraction/reasoning, gpt-5.2 GraphRAG), Microsoft GraphRAG, OpenAI Python SDK, rule-based ml_stub classifier
- **Infrastructure:** Azure VM deployment (screen sessions, ports 8041/8042), Azure Blob Storage, Azure SQL Server (pyodbc, ODBC Driver 18)
- **Databases:** Azure SQL tables dbo.ubo_entities, dbo.ubo_ownership_relationships, dbo.ubo_registry; LanceDB (GraphRAG per-project); Azure Blob JSON/CSV artifacts (risk_assessment_summary.json, missing_info_status.json, risk_decision_status.json, dashboard/metrics/daily_trends.csv)
- **Integrations:** Azure OpenAI, Azure Blob Storage, Azure SQL, SMTP email (missing-info notifications), GraphRAG CLI subprocess

Data Flow

1. Analyst logs into kyc-ui and selects/creates a project_id on Document Upload
2. File (.pdf/.txt/.docx) posts to POST /documents/upload with type and project_id
3. Backend uploads raw bytes to Azure Blob ({project}/{document_id}_{filename})
4. extract_from_blob reads text (PyMuPDF/python-docx), calls LLM extraction agent → structured JSON
5. KYC policy validator checks 25+ required fields; if blocking → status Insufficient Information, writes missing_info_status.json, skips risk scoring
6. If complete → normalizes rows, assigns stable SHA1-based IDs (ENT-*, REL-*, UBO-*), inserts into Azure SQL
7. build_risk_assessment_payload engineers features → ML stub → section scores → Risk Scoring Agent + Reasoning LLM → persists risk_assessment_summary.json and generates DOCX report to blob
8. Document saved to GraphRAG project input; background thread runs graphrag index
9. Dashboard/Queue/Geography endpoints aggregate SQL + blob artifacts with 120s endpoint cache
10. Analyst reviews Risk Assessment, assigns analyst, updates decision (Approved/Escalated/Rejected/Missing Information), optionally sends LLM-drafted email via SMTP
11. Optional: analyst queries indexed documents via POST /chat

External Integrations

- **Azure OpenAI:** Entity extraction, policy validation, email drafting, risk reasoning, GraphRAG LLM/embeddings
- **Azure Blob Storage:** Document storage, KYC result artifacts, dashboard trend CSV
- **Azure SQL Server:** Persistent entities, ownership relationships, UBO registry, document history

- **SMTP:** Outbound missing-information emails (SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL)
- **Microsoft GraphRAG:** Knowledge graph indexing and local/global/drift/basic search over project documents
- **Affine Analytics branding:** Login logo from https://acis.affineanalytics.co.in/assets/images/logo_small.png

KPIs & Metrics Tracked

- **Total Clients in Pipeline:** distinct project_id count from entities table
- **High-Risk Clients:** projects with final_risk_tier = High in persisted summaries
- **Projects Missing Information:** projects with has_missing_info = true in blob status
- **Status Overview:** In-progress / Completed (Approved) / Verification counts from decision blobs
- **Risk Distribution:** Low / Medium / High project counts (donut chart)
- **Onboarding Volume:** daily count of projects with first document upload (30-day rolling UTC trend)
- **Average Processing Time (days):** time from first upload to Approved decision
- **Risk Score (0–100):** final_ai_risk_score_0_100 per project
- **Section Scores (0–100):** geography, ownership_structure, industry_type, sanctions
- **UBO Analysis Score (0–100):** concentration + transparency + complexity + risk_flags components
- **Ownership Complexity:** Simple / Moderate / Complex (derived from ownership section score)
- **Days in Queue:** days since last_uploaded
- **Geographic Exposure:** Countries Represented, High-Risk Jurisdictions (FATF High), Clients in Sanctioned Regions; per-country clientCount, avgRiskScore, fatfRiskRating, sanctions
- **Ownership Distribution:** UBO buckets 0–25%, 25–50%, 50%+ (/dashboard/ownership-distribution)
- **Analytics page (mock/static):** Total cases screened 32,418; Average risk score 41.7; Manual review reduction 52%; Avg onboarding 4.1 → 2.4 days; False positive rate 3.8%

Agentic Flow (if applicable)

Document Upload Trigger → Blob Store → **Entity Extraction Agent** (Azure OpenAI) → **KYC Policy Validator** (Azure OpenAI) → **[Decision: blocking missing fields?]**

→ Yes: persist missing_info_status.json → **Email Drafter** (optional, human sends via SMTP) → **Missing Details UI** (analyst waits for client)

→ No: Azure SQL insert → Feature Engineering → **ML Stub Classifier** → **Risk Scoring Agent + Reasoning LLM** (parallel section narratives) → **UBO Analysis** → persist summary JSON + **DOCX Report Generator** → **GraphRAG**

Index (background)

→ **Dashboard / Client Queue / Risk Assessment UI** → **[Human-in-the-loop: analyst]** assigns analyst, reviews scores/reasoning/ownership graph → Approve | Request More Info | Escalate | Reject → decision blob updated
→ Optional: **GraphRAG Query** (local/global) for document Q&A

Notes & Caveats

- Repo README has unresolved git merge conflict (<<<<<<<< HEAD / >>>>>>> sowhy-new); deployment instructions reference both local (port 8000/5173) and Azure VM (port 8041/8042)
- Hardcoded Azure OpenAI credentials appear in main.py, pipeline_full.py, and risk_scoring_agent.py — production should use .env only
- Login is client-side only (localStorage flag; credentials admin@affine.ai / affine@123) — not backend-authenticated
- Settings page thresholds/weights are UI-local and do not persist to backend scoring logic
- Analytics & Reports page uses static mock data in analytics.ts; report Generate/Download/Schedule buttons are non-functional UI placeholders
- GraphRAG uses separate env vars (AZURE_OPENAI_API_KEY1, AZURE_OPENAI_DEPLOYMENT_NAME1 = gpt-5.2) vs main pipeline (GPT4_LLM_MODEL_DEPLOYMENT_NAME = gpt-5.1)
- Sample client projects in blob/SQL include names like aureline, asteron (Meditech/corporate KYC scenarios)
- Required env vars: AZURE_SQL_*, AZURE_BLOB_*, AZURE_OPENAI_*, GPT4_LLM_MODEL_DEPLOYMENT_NAME, SMTP vars for email, optional KYC_INGESTION_POLICY_PATH

```
{
  "project": {
    "name": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "business_problem": "The client was manually reviewing corporate KYC documents to extract entities, UBOs, and ownership structures, score geographic/industry/sanctions risk, and chase missing policy fields — causing slow onboarding queues, inconsistent scoring, and repetitive follow-up email drafting.",
    "solution_summary": "An agentic KYC pre-screening platform where analysts upload PDF/TXT/DOCX documents, Azure OpenAI extracts and validates structured KYC data against a configurable policy, computes AI risk scores with explainable reasoning, visualizes ownership graphs, generates DOCX reports, manages a prioritized client queue, and indexes documents with GraphRAG for per-project Q&A.",
    "tech_stack": [
      "React 19",
      "TypeScript",
      "Vite",
      "Tailwind CSS",
      "Recharts",
      "D3",

```

```

    "FastAPI",
    "Python",
    "Azure OpenAI",
    "Microsoft GraphRAG",
    "Azure Blob Storage",
    "Azure SQL Server",
    "LanceDB",
    "PyMuPDF",
    "python-docx",
    "SMTP"
  ],
  "outcomes": "Automated entity/UBO extraction from unstructured KYC documents;
policy-gated ingestion blocking incomplete submissions; AI risk scores (0–100) with Low/
Medium/High tiers and section-level reasoning; analyst queue prioritized by risk_score;
downloadable KYC_Risk_Report DOCX; geographic FATF/sanctions exposure dashboard;
LLM-drafted client follow-up emails; GraphRAG document Q&A per project."
},
  "agents": [
    {
      "name": "Entity Extraction Agent",
      "version": "1.0",
      "category": "Document & Data",
      "function_summary": "Extracts entities, ownership relationships, and UBO registry
records from uploaded corporate KYC documents into structured JSON for SQL
persistence.",
      "inputs": [
        "PDF/TXT/DOCX document text",
        "filename",
        "source_document_id"
      ],
      "outputs": [
        "entities array (entity_name, entity_type, jurisdiction, incorporation_date,
registered_address)",
        "ownership_relationships array (parent_entity_id, child_entity_id,
ownership_percentage, control_type)",
        "ubo_registry array (full_name, nationality, ownership_percentage, passport_number,
linked_entities)"
      ],
      "model_used": "Azure OpenAI gpt-5.1 (GPT4_LLM_MODEL_DEPLOYMENT_NAME)",
      "tech_stack": ["Azure OpenAI", "PyMuPDF", "python-docx", "FastAPI"],
      "integrations": ["Azure OpenAI", "Azure Blob Storage", "Azure SQL Server"],
      "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
      "origin_client": "JPMorgan Chase (JPMC)",
      "vertical": "Finance",
      "status": "available",
      "typical_accuracy": null,
      "notes": "Uses JSON response_format; generates deterministic ENT/REL/UBO IDs via
SHA1 when source IDs absent."
    },
    {
      "name": "KYC Policy Completeness Validator",
      "version": "2026-03",

```



```

    "category": "Quality & Compliance",
    "function_summary": "Validates extracted KYC data against kyc_ingestion_policy.json
required fields and blocks risk analysis when mandatory pre-KYC information is missing.",
    "inputs": [
        "extracted JSON from entity extraction",
        "source document text",
        "KYC ingestion policy JSON (25+ required fields)"
    ],
    "outputs": [
        "is_blocking boolean",
        "missing_items list",
        "reason string",
        "extracted_policy_fields dict"
    ],
    "model_used": "Azure OpenAI gpt-5.1",
    "tech_stack": ["Azure OpenAI", "JSON policy engine"],
    "integrations": ["Azure OpenAI", "Azure Blob Storage"],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Deterministic post-validation enforces policy required_if and
must_be_declared rules (e.g., offshore_exposure_declaration)."
},
{
    "name": "Missing Information Email Drafter",
    "version": "1.0",
    "category": "Quality & Compliance",
    "function_summary": "Drafts professional client-facing emails requesting missing KYC
details with structured bullet lists and Compliance Team sign-off.",
    "inputs": [
        "client_name",
        "source_doc",
        "missing_items",
        "reason",
        "missing_item_explanations"
    ],
    "outputs": [
        "subject string",
        "body string (HTML-free plain text email)"
    ],
    "model_used": "Azure OpenAI gpt-5.1",
    "tech_stack": ["Azure OpenAI", "SMTP"],
    "integrations": ["Azure OpenAI", "SMTP email"],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Human-in-the-loop: analyst reviews/edits draft before send via POST /risk-
assessment/request-more-info/send."
}

```

```

    },
    {
      "name": "Main Entity Inference Agent",
      "version": "1.0",
      "category": "Document & Data",
      "function_summary": "Infers the primary corporate entity described in a project's latest document to center the radial ownership visualization.",
      "inputs": [
        "document bytes",
        "filename",
        "candidate entity names from SQL"
      ],
      "outputs": [
        "entity_id",
        "entity_name",
        "reason"
      ],
      "model_used": "Azure OpenAI gpt-5.1",
      "tech_stack": ["Azure OpenAI", "FastAPI"],
      "integrations": ["Azure OpenAI", "Azure Blob Storage", "Azure SQL Server"],
      "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
      "origin_client": "JPMorgan Chase (JPMC)",
      "vertical": "Finance",
      "status": "available",
      "typical_accuracy": null,
      "notes": "Exposed via GET /graph/main-entity; fuzzy-matches LLM output to SQL entity names."
    },
    {
      "name": "ML Risk Classifier Stub",
      "version": "1.0",
      "category": "Quality & Compliance",
      "function_summary": "Rule-based classifier that assigns Low/Medium/High risk tier from engineered geographic, industry, ownership, PEP, and sanctions features.",
      "inputs": [
        "primary_country",
        "fatf_country_risk_band",
        "industry_risk_band",
        "ownership_layers",
        "ownership_opacity",
        "ubo_max_ownership_pct",
        "pep_exposure",
        "sanctions_hits",
        "high_risk_geo_pct",
        "circular_ownership"
      ],
      "outputs": [
        "risk_tier (Low/Medium/High)",
        "confidence (0.74–0.9)"
      ],
      "model_used": "Custom rule-based stub (ml_stub.py)",
      "tech_stack": ["Python"],

```

```

    "integrations": [],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Not a trained ML model; provides gentle prior for section score calibration in
pipeline_full.py."
  },
  {
    "name": "Risk Scoring Agent",
    "version": "1.0",
    "category": "Quality & Compliance",
    "function_summary": "Aggregates geography (30), ownership (30), regulatory/
sanctions (20), and industry (20) scores into a final 0–100 risk score with hard override
rules for FATF blacklist, sanctions, circular ownership, and PEP exposure.",
    "inputs": [
      "RiskFactorScores (geography_score, ownership_score, regulatory_score,
industry_score)",
      "OverrideFlags",
      "client_summary / project_id"
    ],
    "outputs": [
      "final_score (0–100)",
      "risk_tier (Low/Medium/High)",
      "override_triggered",
      "override_reason",
      "reasoning text"
    ],
    "model_used": "Deterministic scoring + Azure OpenAI gpt-5.1 reasoning",
    "tech_stack": ["Azure OpenAI", "Pydantic", "FastAPI"],
    "integrations": ["Azure OpenAI", "Azure Blob Storage"],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Tier cutoffs: 0–33 Low, 34–66 Medium, 67–100 High; overrides force High
tier."
  },
  {
    "name": "Risk Reasoning Generator",
    "version": "1.0",
    "category": "Quality & Compliance",
    "function_summary": "Generates explainable AI narratives for the final risk score and
each of four risk dimensions (Geography, Ownership, Industry, Sanctions) for UI display
and DOCX reports.",
    "inputs": [
      "project_id",
      "section_scores_0_100",
      "final_score",
      "final_tier",

```

```

    "engineered feature summary"
  ],
  "outputs": [
    "final_reasoning (UI paragraph)",
    "report_reasoning (geography, ownership_structure, industry_type, sanctions,
final_summary)"
  ],
  "model_used": "Azure OpenAI gpt-5.1 (AzureOpenAIReasoningLLM)",
  "tech_stack": ["Azure OpenAI", "asyncio"],
  "integrations": ["Azure OpenAI"],
  "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
  "origin_client": "JPMorgan Chase (JPMC)",
  "vertical": "Finance",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Runs 6 parallel LLM prompts; strips parentheses/brackets from output;
required for risk summary payload validity."
},
{
  "name": "UBO Analysis Engine",
  "version": "1.0",
  "category": "Quality & Compliance",
  "function_summary": "Computes UBO-specific risk score from ownership
concentration, identification transparency, structural complexity, and PEP/sanctions/geo
flags.",
  "inputs": [
    "entities from Azure SQL",
    "ubo_registry rows",
    "ownership_relationships"
  ],
  "outputs": [
    "ubo_analysis.score_0_100",
    "ubo_analysis.tier",
    "ubo_analysis.summary",
    "ubo_analysis.metrics (total_ubos, max_ownership_pct, circular_ownership,
pep_count)",
    "ubo_analysis.top_ubos"
  ],
  "model_used": "Deterministic Python (no LLM)",
  "tech_stack": ["Python", "Azure SQL"],
  "integrations": ["Azure SQL Server"],
  "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
  "origin_client": "JPMorgan Chase (JPMC)",
  "vertical": "Finance",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Four 0–25 components: concentration, transparency, complexity, risk_flags."
},
{
  "name": "GraphRAG Index & Query Agent",
  "version": "1.0",
  "category": "Document & Data",

```

```

    "function_summary": "Indexes per-project KYC documents into a knowledge graph
with entity/community extraction and embeddings; answers analyst questions via local,
global, drift, or basic search methods.",
    "inputs": [
        "project_id",
        "TXT documents in projects/{id}/input/",
        "natural language question",
        "search method (local/global/drift/basic)"
    ],
    "outputs": [
        "entities.parquet",
        "community_reports.parquet",
        "LanceDB embeddings",
        "natural language answer text"
    ],
    "model_used": "Microsoft GraphRAG with Azure OpenAI gpt-5.2 + text-embedding-3-
small",
    "tech_stack": ["Microsoft GraphRAG", "LanceDB", "Azure OpenAI", "subprocess"],
    "integrations": ["Azure OpenAI", "Azure Blob Storage"],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Runs as background thread after upload; exposed via POST /chat and
POST /projects/{id}/index."
},
{
    "name": "KYC Risk Report Generator",
    "version": "1.0",
    "category": "Quality & Compliance",
    "function_summary": "Generates downloadable Word (.docx) KYC risk assessment
reports with KPI summary table, section scores, UBO details, override status, and AI
reasoning narratives.",
    "inputs": [
        "risk_assessment_summary.json payload",
        "entities, ubos, relationships from Azure SQL"
    ],
    "outputs": [
        "KYC_Risk_Report_{project}.docx binary"
    ],
    "model_used": "python-docx (no LLM at generation time)",
    "tech_stack": ["python-docx", "FastAPI"],
    "integrations": ["Azure Blob Storage", "Azure SQL Server"],
    "origin_project": "JPMC KYC Risk Pre-Screening (UBO GraphRAG)",
    "origin_client": "JPMorgan Chase (JPMC)",
    "vertical": "Finance",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Threshold bands documented in report: Low 0–30, Medium 31–55, High 56–
100; served via GET /risk-assessment/report."
}

```

```
]
}
```

Affine Analytics VTO Platform — Solution Summary

Client & Context

- **Client:** Affine Analytics (branding: Affine Analytics — VTO Platform, login admin@affine.ai, logo from acis.affineanalytics.co.in, purple #6B2D8B / red #E63946 palette)
- **Vertical:** Retail / CPG (fashion accessories, garments, demographic try-on, catalog categories: Tops, Dresses, Jackets, Accessories, Eyewear, Watches, Jewelry)
- **Project Type:** Agentic System + Operations Dashboard (AI multi-stage try-on pipeline with React admin UI)

Business Problem

Retail and brand teams were manually compositing customer photos with product shots (or relying on static mannequin imagery), which was slow, inconsistent across accessories, and hard to scale for multi-item looks (jacket + bag + hat in one frame). That manual workflow caused long turnaround times, weak personalization, and no unified way to browse a garment library and rerun past generations.

Solution Overview

Affine Analytics built a **Virtual Try-On (VTO) Platform** that lets users upload a person photo plus up to eight accessory/garment images and receive a single photorealistic composite. The React frontend (vto-platform, Vite on port **8045**) proxies /api to a FastAPI backend on port **8046**. The backend validates and preprocesses images, uses **Azure OpenAI GPT-4o** (vision) to author a precise image-editing prompt, then **Google Gemini** (image output modality) to synthesize the try-on. Successful runs persist to ./data/sessions with assets and metadata. A **Garment Catalog** loads accessories from **Azure Blob Storage** (SAS URLs), supports upload, and optional **semantic search** via **Azure AI Vision** embeddings + **Azure AI Search** vector index vto-accessories. Session History reads real API data; Analytics and Pipeline Monitor pages use **mock/demo data** for KPIs and ops views.

Key Features

- **Try-On Studio** — upload person photo + multi-accessory uploader (max 8); Generate Try-On POST to /api/v1/tryon/generate; shows staged progress, output preview, download PNG, regenerate
- **Catalog → Studio handoff** — import selected catalog accessories into Try-On Studio via blob proxy /api/v1/catalog/blob
- **Garment Catalog** — list Azure blob accessories, text filter, semantic vector search (/api/v1/catalog/search), multi-select, upload to catalog, “Try on with selected” / “Add to Try-On”
- **Session History** — list persisted sessions from /api/v1/tryon/sessions, search by ID/prompt, modal with person/accessories/output images and GPT placement prompt

- **Analytics Dashboard** — metric cards and Recharts (daily sessions, category breakdown, demographic weekly, latency trend, category performance table) from src/mockData/analytics.ts
- **Pipeline Monitor** — mock service health, live-style log stream, architecture step cards, throughput chart, recent errors table from src/mockData/pipeline.ts
- **Email/password login** — client-side gate (admin@affine.ai / affine@123), session in localStorage
- **Collapsible sidebar navigation** — Try-On Studio, Session History, Analytics, Catalog, Pipeline Monitor
- **Toast notifications** — success/error/info feedback across flows
- **Health check API** — /api/v1/health probes Azure OpenAI and Gemini reachability
- **Swagger docs** — FastAPI auto docs at /docs
- **Keyboard shortcut** — ⌘/Ctrl+Enter to generate in Try-On Studio
- **Export CSV** (Session History UI button — present in UI, not wired to backend export endpoint)

AI Agents & Components

GPT-4o Fashion Vision Analyzer (gpt_analyzer.py)

- **Function:** Multimodal analysis of person + ordered garment/accessory images; produces one paragraph image-editing instruction for virtual try-on (pose, skin tone, layering, lighting, framing preservation)
- **Input:** Preprocessed JPEG person bytes; list of garment JPEGs; optional demographic, category, style_tags form fields
- **Output:** prompt string; duration_ms; tokens_used
- **Model/Service:** Azure OpenAI — deployment from AZURE_OPENAI_DEPLOYMENT (example: gpt-4o); AsyncAzureOpenAI; temperature 0.2
- **Position in flow:** Processing (stage gpt_analysis after input_validation)

Gemini Image Compositor (gemini_compositor.py)

- **Function:** Image synthesis using GPT-generated editing prompt; applies all garments in one output; retries up to 3 times with exponential backoff
- **Input:** Person JPEG bytes; garment JPEG list; editing_prompt string
- **Output:** output_image_bytes; mime_type; duration_ms
- **Model/Service:** Google GenAI — settings.GEMINI_MODEL (config default gemini-3-pro-image-preview; .env.example documents models/gemini-2.0-flash-exp); response_modalities=["IMAGE"]; aspect ratio matched via nearest_gemini_aspect_ratio
- **Position in flow:** Processing (stage gemini_composition)

Azure AI Vision Embedding Service (vision_embeddings.py)

- **Function:** vectorizeImage / vectorizeText via Azure Computer Vision retrieval APIs (1024-dim vectors)
- **Input:** Raw image bytes or natural-language query string
- **Output:** list[float] embedding vector

- **Model/Service:** Azure AI Vision
— AZURE_VISION_ENDPOINT, AZURE_VISION_KEY, model version 2023-04-15
- **Position in flow:** Ingestion/indexing on catalog upload; processing on semantic search query

Azure AI Search Catalog Vector Index (catalog_search_index.py)

- **Function:** HNSW cosine vector search over accessory catalog documents (id, blob_name, name, category, tags, embedding)
- **Input:** Query embedding vector; top (1–48)
- **Output:** Ranked hits with @search.score
- **Model/Service:** Azure AI Search index vto-accessories (auto-created on startup if configured)
- **Position in flow:** Retrieval for Garment Catalog semantic search; upsert on upload when Vision+Search configured

Input Validation & Preprocessing (image_utils.py — non-LLM)

- **Function:** Size/format/dimension checks; convert to RGB JPEG; max dimension 2048; quality 90
- **Input:** Raw upload bytes (max MAX_IMAGE_SIZE_MB, default 10 MB)
- **Output:** Normalized JPEG bytes
- **Model/Service:** Pillow (PIL)
- **Position in flow:** Ingestion (input_validation)

Tech Stack

- **Frontend:** React 18, TypeScript, Vite 5, React Router 6, Tailwind CSS 3, Recharts 2, Lucide React, Sora font (Google Fonts)
- **Backend:** Python 3.11+, FastAPI, Uvicorn, Pydantic v2, python-multipart, Pillow, httpx, asyncio thread offloading
- **AI/ML:** Azure OpenAI (GPT-4o vision chat), Google Gemini (google-genai), Azure AI Vision multimodal embeddings, Azure AI Search vector HNSW
- **Infrastructure:** Local dev — Vite :8045 proxying to FastAPI :8046; filesystem session store (SESSIONS_DIR); temp files (TEMP_DIR)
- **Databases:** No SQL DB — JSON meta.json per session under ./data/sessions/VTO-XXXXXX/; Azure Blob for catalog images; Azure AI Search for vectors
- **Integrations:** Azure OpenAI, Google AI Studio (Gemini), Azure Blob Storage (accessories container), Azure AI Vision, Azure AI Search; Affine logo CDN [acis.affineanalytics.co.in](https://cdn.acis.affineanalytics.co.in)

Data Flow

1. User authenticates on Login page → email stored in localStorage via persistAuth
2. User uploads **person_image** and 1–8 **garment_images** in Try-On Studio (or imports catalog blobs as File objects)
3. Frontend POST multipart/form-data to /api/v1/tryon/generate with optional demographic, category, style_tags
4. Backend assigns session_id (VTO- + 6 alphanumeric), reads file bytes
5. **input_validation:** parallel validate_and_preprocess on all images → JPEG normalization

6. **gpt_analysis:** base64 vision messages to Azure OpenAI → single editing prompt
7. **gemini_composition:** person + garments + prompt → Gemini image output bytes
8. **output_processing:** save PNG to TEMP_DIR/{session_id}_output.png; on success **persist** person.jpg, accessory_{i}.jpg, output.png, meta.json under SESSIONS_DIR
9. API returns TryOnResponse with output_image_base64, pipeline_stages timings, metadata.total_duration_ms, accessory_count
10. UI displays composite; refreshSessions loads history from /api/v1/tryon/sessions
11. **Catalog path:** blobs listed with SAS URLs → optional text embedding search → user selects items → blob fetched via /api/v1/catalog/blob → appended to studio uploader
12. **Catalog upload path:** POST /api/v1/catalog/accessories/upload → Azure Blob uploads/{uuid}_{filename} → optional Vision embed + Search upsert

External Integrations

- **Azure OpenAI:** GPT-4o vision prompt generation; health probe
- **Google Gemini API:** Photorealistic try-on image generation with IMAGE response modality
- **Azure Blob Storage:** Accessory catalog storage, SAS read URLs, same-origin blob proxy for browser File construction
- **Azure AI Vision:** vectorizeImage / vectorizeText for 1024-d embeddings
- **Azure AI Search:** Vector index vto-accessories for semantic catalog retrieval
- **Affine ACIS CDN:** Login page logo asset

KPIs & Metrics Tracked

- **Total Try-On Sessions:** 2,847 (+18.4% vs last month) — mock Analytics Dashboard
- **Success Rate:** 94.2% (+2.1%) — mock
- **Avg. Realism Score:** 88.7% (+3.2%) — mock (schema supports realism_score but pipeline does not compute it today)
- **Avg. Pipeline Duration:** 7.3s (-1.2s improvement) — mock; live sessions expose pipeline_duration_s / total_duration_ms per run
- **Daily sessions chart:** total vs successful per day (30 days) — mock DAILY_SESSIONS
- **Category breakdown:** Tops 28%, Dresses 22%, Jackets 18%, Accessories 14%, Eyewear 10%, Watches 5%, Jewelry 3% — mock
- **Demographic weekly:** man, woman, child session counts — mock
- **Latency trend:** avgLatency by day — mock
- **Category performance table:** sessions, avg score, success rate, trend per category — mock
- **Pipeline Monitor live stats:** Active Sessions, Queue Depth, GPU Utilization %, Memory Usage %, API Calls (1hr), Error Rate % — simulated client-side

- **Per-session UI:** pipeline duration (seconds), accessory count, status badge (Success/Failed)
- **Semantic search:** cosine-related score per catalog hit

Agentic Flow (if applicable)

Trigger: User clicks **Generate Try-On** (or ⌘+Enter) with person + ≥ 1 accessory loaded

- **Input Validation** (Pillow preprocess, fail fast INVALID_IMAGE)
- **GPT-4o Fashion Vision Analyzer** (build placement prompt; fail GPT_ANALYSIS_ERROR)
- **Gemini Image Compositor** (synthesize image; up to 3 retries; fail GEMINI_EMPTY_RESPONSE)
- **Output Processing** (write temp PNG; fail OUTPUT_WRITE_ERROR)
- **[Decision]** status === success → persist session assets + meta.json; else return stage-specific error
- **Human-in-the-loop:** review output in studio; Download / Regenerate / navigate to Session History for prompt + inputs
- **Optional parallel catalog flow:** natural-language search → Vision text embed
- Search vector hits → human selects garments → blob download → same try-on trigger

Note: UI shows a 4th “Quality Validation” step; backend pipeline has four stages but validation is not a separate ML service—
only input_validation, gpt_analysis, gemini_composition, output_processing.

Notes & Caveats

- **Model naming in UI vs config:** Frontend labels “GPT 5.4” and gemini-3-pro-image-preview; README/.env.example reference gpt-4o and gemini-2.0-flash-exp; persisted sessions store GPT 5.4 + gemini-3-pro-image-preview
- **Analytics & Pipeline Monitor** use src/mockData/* — not connected to live telemetry APIs
- **Realism score / SSIM / quality validator** referenced in mock pipeline monitor but not implemented in run_tryon_pipeline
- **Login** is hardcoded client-side demo auth, not backend OAuth
- **Required env:** AZURE_OPENAI_*, GEMINI_API_KEY; catalog needs AZURE_STORAGE_CONNECTION_STRING; semantic search needs AZURE_VISION_* + AZURE_SEARCH_*
- **Gemini image output** requires API tier with response_modalities=["IMAGE"] per backend README
- **Max constraints:** 10 MB per image, 8 accessories, images min 100×100, formats JPG/PNG/WEBP
- **CORS:** dev uses Vite proxy (API_BASE empty in DEV); production may set VITE_API_BASE
- **Repo path hint:** README references AI CoE Solutions/VirtualTryOn/virtual-tryOn/ deployment layout

```
{
  "project": {
```

```

"name": "Affine Analytics VTO Platform",
"client": "Affine Analytics",
"vertical": "Retail / CPG",
"business_problem": "Retail teams were manually compositing customer photos with
multiple garment and accessory images, which was slow, inconsistent for layered looks,
and did not scale with a centralized product catalog or repeatable session history.",
"solution_summary": "A React operations UI and FastAPI backend that run a multi-stage
virtual try-on pipeline: validate images, use Azure OpenAI GPT-4o vision to author a
placement prompt, synthesize the composite with Google Gemini image generation,
persist successful sessions to disk, and optionally manage an Azure Blob accessory
catalog with Azure AI Vision + Azure AI Search semantic retrieval.",
"tech_stack": [
  "React 18",
  "TypeScript",
  "Vite 5",
  "Tailwind CSS",
  "Recharts",
  "FastAPI",
  "Python 3.11",
  "Azure OpenAI GPT-4o",
  "Google Gemini",
  "Azure Blob Storage",
  "Azure AI Vision",
  "Azure AI Search",
  "Pillow"
],
"outcomes": "Single-click photorealistic multi-accessory try-on from person + up to 8
product images; persisted session history with prompts and assets; catalog browse/upload
with optional natural-language semantic search; operational views for adoption and
pipeline health (analytics/pipeline pages currently demo/mock data).",
},
"agents": [
{
  "name": "GPT-4o Fashion Vision Analyzer",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Analyzes person and garment reference images with a fashion/
computational-photography system prompt and outputs a single precise image-editing
instruction for virtual try-on, including layering, lighting, and full-frame preservation.",
  "inputs": [
    "person_image (JPEG bytes)",
    "garment_images (1-8 JPEG bytes)",
    "demographic (optional form field)",
    "category (optional form field)",
    "style_tags (optional comma-separated form field)"
  ],
  "outputs": [
    "generated_prompt (string)",
    "duration_ms",
    "tokens_used"
  ],
},
"model_used": "Azure OpenAI GPT-4o (AZURE_OPENAI_DEPLOYMENT)",

```

```

    "tech_stack": [
      "Azure OpenAI",
      "openai Python SDK",
      "FastAPI",
      "base64 vision messages"
    ],
    "integrations": [
      "Azure OpenAI"
    ],
    "origin_project": "Affine Analytics VTO Platform",
    "origin_client": "Affine Analytics",
    "vertical": "Retail / CPG",
    "status": "live",
    "typical_accuracy": null,
    "notes": "Stage gpt_analysis in pipeline; temperature 0.2; UI may label model as GPT
5.4."
  },
  {
    "name": "Gemini Image Compositor",
    "version": "1.0",
    "category": "Sales & Revenue",
    "function_summary": "Synthesizes the final virtual try-on image from person photo,
accessory references, and the GPT-generated editing prompt using Gemini image output
modality with aspect ratio matching and up to three retries.",
    "inputs": [
      "person_image (JPEG bytes)",
      "garment_images (JPEG bytes list)",
      "editing_prompt (string from GPT analyzer)"
    ],
    "outputs": [
      "output_image_bytes",
      "mime_type",
      "duration_ms"
    ],
    "model_used": "Google Gemini (GEMINI_MODEL, e.g. gemini-3-pro-image-preview or
gemini-2.0-flash-exp)",
    "tech_stack": [
      "google-genai",
      "FastAPI",
      "Pillow aspect ratio helper"
    ],
    "integrations": [
      "Google AI Studio / Gemini API"
    ],
    "origin_project": "Affine Analytics VTO Platform",
    "origin_client": "Affine Analytics",
    "vertical": "Retail / CPG",
    "status": "live",
    "typical_accuracy": null,
    "notes": "Stage gemini_composition; requires IMAGE response_modalities support on
API tier."
  },

```

```

{
  "name": "Azure AI Vision Catalog Embedder",
  "version": "1.0",
  "category": "Document & Data",
  "function_summary": "Generates 1024-dimensional multimodal embeddings for catalog
accessory images and natural-language search queries to power semantic garment
discovery.",
  "inputs": [
    "image_bytes (catalog upload or index job)",
    "text query string (semantic search)"
  ],
  "outputs": [
    "embedding vector (list[float], 1024 dims)"
  ],
  "model_used": "Azure AI Vision retrieval APIs (vectorizeImage / vectorizeText, model-
version 2023-04-15)",
  "tech_stack": [
    "Azure AI Vision",
    "httpx"
  ],
  "integrations": [
    "Azure AI Vision"
  ],
  "origin_project": "Affine Analytics VTO Platform",
  "origin_client": "Affine Analytics",
  "vertical": "Retail / CPG",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Optional; enabled when AZURE_VISION_ENDPOINT and
AZURE_VISION_KEY are set."
},
{
  "name": "Azure AI Search Catalog Vector Retrieval",
  "version": "1.0",
  "category": "Document & Data",
  "function_summary": "Maintains and queries the vto-accessories HNSW vector index
for cosine-similarity ranking of catalog items from text or image-derived embeddings.",
  "inputs": [
    "query embedding vector",
    "top (1-48)"
  ],
  "outputs": [
    "ranked catalog documents (id, blob_name, name, category, tags)",
    "search score"
  ],
  "model_used": "Azure AI Search HNSW (cosine, 1024-d embedding field)",
  "tech_stack": [
    "azure-search-documents",
    "Azure AI Search"
  ],
  "integrations": [
    "Azure AI Search",

```

```

    "Azure Blob Storage (SAS URL enrichment for hits)"
  ],
  "origin_project": "Affine Analytics VTO Platform",
  "origin_client": "Affine Analytics",
  "vertical": "Retail / CPG",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Index auto-ensured on startup; optional alongside Vision embedder; endpoint
GET /api/v1/catalog/search."
}
]
}

```

Aira (CreAltive Studio) — Solution Summary

Client & Context

- **Client:** Staples (Staples-Creative repo; Staples-style Amazon PDP workflows), delivered in an **Affine Analytics** workspace branded as **Aira**
- **Vertical:** Retail / CPG (Amazon Product Detail Page creative production)
- **Project Type:** Agentic System + creative studio dashboard (React UI + multi-step AI orchestration backend)

Business Problem

The client was manually producing Amazon PDP image sets and short product videos from reference photography, product specification PDFs, and marketplace guardrails PDFs—a process that required designers to read long spec/guideline documents, craft role-specific prompts per slot (Main Image, Lifestyle, Infographic, Dimensions, etc.), run separate image tools, and separately verify marketplace compliance. That manual loop caused slow turnaround, inconsistent SKU fidelity across variants, and rework when creatives violated guardrails or mis-stated specs on-pack.

Solution Overview

Aira is a full-stack creative studio: users authenticate into a project workspace, upload up to four reference images plus optional **product specification** and **guardrails** PDFs, choose PDP image types and an image model (Gemini Flash or Azure GPT Image 1.5), and generate up to six variants in parallel. The FastAPI backend extracts unlimited PDF text (PyMuPDF), builds long prompts embedding verbatim excerpts (with configurable truncation), optionally refines them with **Azure OpenAI gpt-5.4** vision+text, then calls **Gemini image** or **Azure images/edits** per variant. Users review outputs in the Editor, run **Gemini vision compliance** scoring against guardrails only, export PNG/JPEG/SVG, iterate via session **edit-image** flows, or switch to **Sora-2** image-to-video with an Azure-authored script prompt. When Azure Blob is configured, projects, sessions, library assets, and compliance reports persist for History and rerun.

Key Features

- **Project workspaces** — create/rename/delete projects; each holds sessions, uploads, and generated assets (/home, ProjectRecord)

- **Session studio (Editor)** — per-session name, reference image upload (max 4), prompt text, image-type multi-select, model picker, generate flow
- **PDP image type slots** — Main Image, Lifestyle, Infographic, Multi Angles, Dimensions, Closeup (Packaging/Assembly defined in backend but commented out in UI)
- **Product spec PDF upload** — unlocks Infographic & Dimensions types; required for full six outputs and spec-grounded dimension callouts
- **Guardrails PDF upload** — tightens generation prompts; **both** spec + guardrails required for compliance scoring
- **Dual image providers** — gemini-flash (Gemini 2.5 Flash Image) and gpt-image (Azure GPT Image 1.5, fast/quality speed mode)
- **Parallel multi-variant generation** — 1–6 images per request with per-slot image_types, partial-failure reporting (failed_variants, warnings, meta.prompt_source)
- **Azure prompt author (optional)** — gpt-5.4 vision rewrites per-role prompts from references + PDF excerpts; falls back to template + per-role directives
- **Template prompt engine** — PromptGenerator embeds SKU fidelity rules, cross-image-type constraints, and PDF excerpts into each variant prompt
- **Compliance check** — per generated image: score 0–100, passed_rules, failed_rules, warnings, suggestions in ComplianceModal / ComplianceBadge
- **Compliance scope lock** — product spec PDF validated on upload but **not** sent to compliance model; only guardrails excerpt drives scoring
- **Image export** — download current result as JPEG, JPG, PNG, or SVG from Editor/History
- **Result carousel** — keyboard ArrowLeft/ArrowRight navigation across generated outputs
- **Estimated generation cost** — UI shows effectiveCount × model.costPerUnit (40 for Gemini, 60 for GPT Image)
- **Credit balance pill** — workspace credits display (mock creditData: balance 1240 / total 5000 / used 3760)
- **Image resolution presets (display)** — HD 1280×720 through 8K 7680×4320 labels in Editor (marketing labels; actual provider output follows model)
- **Video generation mode** — toggle Image vs Video; Sora constraints 1280×720 or 720×1280, 4/8/12 seconds
- **Video prompt preview** — POST /api/video/prompt builds Amazon PDP video script via Azure gpt-5.4 before render
- **Sora image-to-video** — approved prompt + reference image → MP4 base64 (sora-2 deployment)
- **Session persistence** — POST /projects/{id}/sessions runs full pipeline and stores inputs/outputs/PDFs in Azure Blob (creative-assets container default)
- **Session image edit** — edit-image with modes add/replace, optional image_type, re-runs generation on one output
- **Project source library** — deduplicated blob-backed library (max 200 items, batch upload 20) reusable across sessions

- **Global & project History** — list sessions, view details modal, rerun into Editor via rerunBridge, delete sessions
- **Rehydrate saved sessions** — reopen Editor with prior images, videos, compliance cache, PDFs
- **PDF upload to existing session** — POST .../sessions/{id}/pdfs attaches spec/guardrails after the fact
- **Static asset streaming** — GET /projects/{id}/sessions/{id}/{inputs|outputs}/{name} and library URLs
- **Light/dark UI theme** — creative-ui-theme in localStorage, sidebar toggle
- **Demo authentication** — email/password gate (RequireAuth); SSO button shows “not configured” toast
- **Health endpoint** — GET /api/health for API readiness
- **Toast notifications** — success/error/info for generation, compliance, SSO, etc.

AI Agents & Components

Generation Prompt Author

- **Function:** Vision+text LLM authors JSON array of per-image_type multimodal prompts (or per-role directive paragraphs for template fallback) from reference photos + PDF excerpts + user notes
- **Input:** Up to 4 InputImage buffers, image_types[], generic_prompt, PromptDocumentExcerpts (product spec + guardrails text, truncated)
- **Output:** list[GeneratedPrompt] with image_type + long prompt string, or role_directives map
- **Model/Service:** Azure OpenAI Chat Completions — deployment AZURE_OPENAI_PROMPT_DEPLOYMENT (default **gpt-5.4**); enabled via GEMINI_PROMPT_AUTHOR env flag (legacy name)
- **Position in flow:** Processing — after PDF extraction, before parallel image provider calls

Prompt Generator (template engine)

- **Function:** Deterministic prompt assembly with embedded PRODUCT_COPY_AND_FACT_RULES, REFERENCE_SKU_FIDELITY_RULES, CROSS_IMAGE_TYPE_CONSTRAINTS, role-specific _IMAGE_TYPE_GUIDANCE, and verbatim PDF excerpt blocks
- **Input:** Scheduled image_types, user prompt, PromptDocumentExcerpts, optional Azure per_role_directives
- **Output:** GeneratedPrompt[] used as provider prompts
- **Model/Service:** No LLM (static templates); fed by PDF text from PdfExtractor
- **Position in flow:** Processing — fallback or hybrid when Azure prompt author unavailable

PdfExtractor

- **Function:** Extracts all selectable text from uploaded PDF pages (PyMuPDF / pypdf path in utils)
- **Input:** product_spec_pdf and/or guardrails_pdf bytes

- **Output:** Plain-text strings; later truncated by GENERATION_PDF_SPEC_MAX_CHARS / GENERATION_PDF_GUARDRAILS_MAX_CHARS (default 32,000 each)
- **Model/Service:** PyMuPDF
- **Position in flow:** Ingestion → processing

Gemini Image Provider

- **Function:** Multimodal image generation — reference images + text prompt → one PNG per variant
- **Input:** ProviderGenerateRequest (images, prompt, variant index/label)
- **Output:** PNG bytes per variant (images_b64 in API response)
- **Model/Service:** Google **Gemini** via google-genai — default GEMINI_MODEL=gemini-2.5-flash-image (override via form model)
- **Position in flow:** Output — parallel fan-out (up to MAX_PARALLEL_WORKERS=4)

Azure OpenAI Image Provider

- **Function:** Product image edit/generation via Azure images/edits multipart API
- **Input:** First reference image PNG + prompt per variant
- **Output:** Base64 PNG from Azure response
- **Model/Service:** Azure OpenAI deployment AZURE_OPENAI_IMAGE_MODEL (default **gpt-image-1.5**); auth bearer or api_key; speed AZURE_OPENAI_IMAGE_SPEED_MODE fast|quality
- **Position in flow:** Output — parallel fan-out (alternative to Gemini)

Compliance Checker

- **Function:** Vision LLM reviews one generated PDP image against guardrails excerpt only; returns structured compliance JSON
- **Input:** Generated image file, image_type string, guardrails PDF text (type-specific excerpt via compliance_rules_engine.build_rules_context, max ~12k chars)
- **Output:** compliance_score (0–100, recalculated from pass/fail counts when possible), passed_rules, failed_rules, warnings, suggestions
- **Model/Service:** Google Gemini — GEMINI_COMPLIANCE_MODEL (default **gemini-2.5-flash**), response_mime_type=application/json, temperature 0.35, retry without JSON MIME on parse failure
- **Position in flow:** Validation — human-triggered post-generation in UI (POST /api/check-compliance)

Compliance Rules Engine

- **Function:** Parses guardrails PDF text into image-type-focused excerpts (section tokens for Main Image, Lifestyle, Infographic, etc.) before compliance LLM call
- **Input:** Raw guardrails text + normalized image_type
- **Output:** ComplianceRulesContext with guideline_excerpt
- **Model/Service:** Rule-based text segmentation (no LLM)
- **Position in flow:** Processing — immediately before Compliance Checker

Video Prompt Service

- **Function:** Builds full Amazon PDP video ad script (product reference block, compliance checklist table, creative direction, scene script + generation prompts) from spec/guardrails/user script
- **Input:** Product spec text (trim 2500 chars), guardrails text (trim 2500), optional user_script_or_prompt, reference image bytes, width/height/seconds
- **Output:** Long text prompt string (clamped to 15,000 UTF-8 bytes for Sora)
- **Model/Service:** Azure OpenAI **gpt-5.4** (AZURE_OPENAI_PROMPT_DEPLOYMENT) chat with vision when reference image supplied
- **Position in flow:** Processing — before Sora submit (or skipped if user supplies approved_prompt)

Sora Video Service

- **Function:** Image-to-video job orchestration on Azure OpenAI Videos API — create job → poll → download MP4
- **Input:** Clamped prompt, reference image resized to 1280×720 or 720×1280, seconds ∈ {4,8,12}
- **Output:** MP4 bytes (base64 in API)
- **Model/Service:** Azure OpenAI deployment AZURE_OPENAI_VIDEO_DEPLOYMENT (default **sora-2**)
- **Position in flow:** Output — after prompt approval in Editor

Video Generation Orchestrator

- **Function:** Coordinates PDF extraction → dynamic prompt (or approved prompt) → Sora pipeline; attaches meta (truncation, char counts)
- **Input:** Same as video endpoints on video_generate.py
- **Output:** VideoGenerateResponse (video_b64, prompt_used, meta)
- **Model/Service:** Composes VideoPromptService + SoraVideoService
- **Position in flow:** End-to-end video path

Session Service

- **Function:** Persists inputs/outputs/PDFs to Azure Blob, runs same GenerationService as stateless /api/generate, stores compliance_reports on session JSON, increments project.experiments
- **Input:** Multipart session create / edit-image / PDF upload
- **Output:** SessionRecord / SessionPayload with blob URLs
- **Model/Service:** Azure Blob Storage (azure-storage-blob)
- **Position in flow:** Persistence layer around generation and compliance

Tech Stack

- **Frontend:** React 18, TypeScript, Vite 8, React Router 6, Tailwind CSS 3, PostCSS, DM Sans + Space Mono (Google Fonts), native fetch multipart APIs
- **Backend:** Python 3, FastAPI, Uvicorn, Pydantic dataclasses/models, python-multipart, Pillow, asyncio parallel utils (run_blocking_parallel_async, retries)

- **AI/ML:** Google Gemini (google-genai) for image gen + compliance vision; Azure OpenAI (openai SDK patterns + requests) for gpt-5.4 prompts, gpt-image-1.5 edits, Sora-2 video
- **Infrastructure:** Azure OpenAI (Cognitive Services endpoint pattern), Azure Blob Storage container creative-assets; local dev CORS to localhost:5173; optional backend/.env + backend/key.py fallback
- **Databases:** None — metadata as JSON (project.json, session.json) in blob storage; no SQL/ORM
- **Integrations:** Google Gemini API, Azure OpenAI (chat, images/edits, videos), Azure Blob Storage

Data Flow

Image generation (stateless or session-backed):

1. User uploads 1–4 reference images + optional product spec PDF + guardrails PDF + prompt + image_types[] + provider_id via Editor → POST /api/generate or POST /projects/{id}/sessions
2. Backend validates/normalizes images (load_and_validate_uploads, max 12 MB each) and reads PDF bytes
3. PdfExtractor extracts full text from both PDFs
4. PromptGenerator.prepare_pdf_excerpts_for_prompts truncates to GENERATION_PDF_*_MAX_CHARS and emits truncation warnings
5. GenerationPromptAuthor.try_author_prompts (if enabled) calls Azure gpt-5.4 with images + excerpts → per-variant prompts; else template prompts (+ optional per-role directives)
6. GenerationService schedules num_images variants mapped to image_types (cyclic if fewer types than count)
7. Parallel worker pool calls GeminiProvider or AzureOpenAIImageProvider per variant with retries/timeouts
8. format_outcome returns base64 PNGs + meta (prompt_source, labels, image_types) + warnings/failed_variants
9. UI renders data URLs in results panel; optional blob persist on session path

Compliance:

1. User clicks Compliance on a generated image (requires both PDFs uploaded for session)
2. POST /api/check-compliance with image file + both PDFs + image_type
3. Guardrails text extracted; build_rules_context selects type-relevant excerpt
4. ComplianceChecker sends image + instruction to Gemini compliance model → JSON score/lists
5. UI shows ComplianceBadge and ComplianceModal; session path caches under compliance_reports[imageUrl]

Video:

1. User selects Video mode, one reference image, product spec PDF, optional guardrails + script
2. POST /api/video/prompt → VideoPromptService (Azure gpt-5.4) → editable videoPromptDraft
3. User approves prompt → POST /api/video/generate or generate-and-save with approved_prompt

4. VideoGenerationOrchestrator clamps prompt → SoraVideoService submit/poll/download
5. Editor displays video/mp4 data URL; optional blob save on project session

External Integrations

- **Google Gemini API:** PDP image generation (GEMINI_MODEL) and compliance vision (GEMINI_COMPLIANCE_MODEL)
- **Azure OpenAI:** gpt-5.4 prompt authoring (AZURE_OPENAI_PROMPT_DEPLOYMENT), gpt-image-1.5 image edits (AZURE_OPENAI_IMAGE_MODEL), Sora-2 video (AZURE_OPENAI_VIDEO_DEPLOYMENT)
- **Azure Blob Storage:** Project/session/input/output/library asset persistence (AZURE_STORAGE_*, container creative-assets)
- **Google Fonts CDN:** Typography for studio UI

KPIs & Metrics Tracked

- **compliance_score:** 0–100 marketplace guardrail adherence per generated image (color thresholds: ≥85 green, ≥60 amber, else red in UI)
- **passed_rules / failed_rules:** Countable guardrail criteria with observations tied to pixels
- **warnings / suggestions:** Borderline issues and fix recommendations from compliance model
- **project.images:** Total images counter on project record (blob metadata)
- **project.experiments:** Incremented per session generation run
- **project.credits:** Credit counter on project record (alongside global mock credit balance in UI)
- **Sessions count:** Per-project session list length on Project Sessions page
- **estimatedCost:** num_images × model.costPerUnit (40 Gemini / 60 GPT Image) shown before generate
- **Workspace credits:** 1240 balance, 5000 total, 3760 used (static creditData in frontend)
- **Session status:** pending | running | success | partial | error on SessionRecord
- **Generation**
meta: prompt_source (e.g. azure_gpt_prompt_author, template_with_full_excerpts), truncation warnings, failed_variants, duration_ms per variant
- **Video**
meta: prompt_truncated, prompt_bytes_sent, product_spec_chars, guardrails_chars, used_approved_prompt

Agentic Flow (if applicable)

Image generation pipeline:

User clicks **Generate** → PdfExtractor → [GenerationPromptAuthor Azure gpt-5.4 **OR** PromptGenerator template] → **[parallel branch per image_type slot]** → GeminiProvider **OR** AzureOpenAIImageProvider → base64 PNGs to UI → (optional human) **Compliance**

check → ComplianceRulesEngine → ComplianceChecker (Gemini) → score + rule lists → user export or **edit-image** (re-enter generation on one slot)

Video pipeline:

User selects Video → uploads image + spec PDF → **Generate**

prompt → VideoPromptService (Azure gpt-5.4) → human edits/
approves videoPromptDraft → **Generate**

video → VideoGenerationOrchestrator → SoraVideoService (poll job) → MP4 in results viewer

Session persistence branch: Same generation/compliance flows but SessionService writes inputs/outputs/PDFs to Azure Blob before/after API response so History and rerun work offline from browser memory.

Notes & Caveats

- Configure backend/.env per backend/app/core/config.py (Gemini **or** Azure image provider required at startup; blob storage optional but needed for projects API).
- GEMINI_PROMPT_AUTHOR env name is legacy — prompt authoring runs on **Azure gpt-5.4**, not Gemini.
- Without product spec PDF, only **Main Image, Lifestyle, Multi Angles, Closeup** are allowed; max **4** outputs (vs 6 with spec).
- Compliance intentionally **ignores product spec content** in the model prompt (spec PDF still required on upload for API symmetry).
- Sora prompts hard-capped at **15,000 UTF-8 bytes** (SORA_IMAGE_TO_VIDEO_PROMPT_MAX_BYTES).
- Video resolution limited to **1280×720** or **720×1280** at API level regardless of UI “4K/8K” labels on image mode.
- Login is **demo credentials only**; SSO not wired.
- Do not commit secrets — use .env / key.py (gitignored pattern expected).
- Run API: uvicorn backend.main:app --reload --host 127.0.0.1 --port 8001; frontend VITE_API_BASE_URL points to backend.

Agent catalog JSON

```
{
  "project": {
    "name": "Aira (CreAltive Studio)",
    "client": "Staples",
    "vertical": "Retail / CPG",
    "business_problem": "The client was manually creating Amazon PDP image sets and product videos from reference photos, product specification PDFs, and marketplace guardrails—reading long documents, writing per-slot creative briefs, running separate image tools, and reworking assets that failed marketplace compliance.",
    "solution_summary": "Aira is a Staples-oriented Amazon PDP creative studio: users upload references and PDFs, generate up to six role-specific images in parallel via Gemini or Azure GPT Image with optional gpt-5.4 prompt authoring, score outputs against guardrails with Gemini vision compliance, and produce Sora-2 product videos from Azure-authored scripts—with optional Azure Blob persistence for projects and sessions.",
    "tech_stack": [
      "React 18 + TypeScript + Vite",
      "Tailwind CSS",
      "FastAPI + Uvicorn",
      "Google Gemini (google-genai)",
    ]
  }
}
```

```

    "Azure OpenAI (gpt-5.4, gpt-image-1.5, sora-2)",
    "Azure Blob Storage",
    "PyMuPDF",
    "Pillow"
  ],
  "outcomes": "Faster PDP variant production grounded in PDF spec/guardrail text,
parallel multi-slot generation, quantified compliance scores (0–100) with pass/fail rule lists,
and scripted Sora video output—reducing manual prompt writing and compliance rework."
},
  "agents": [
    {
      "name": "Generation Prompt Author",
      "version": "2.1.0",
      "category": "Sales & Revenue",
      "function_summary": "Azure vision+text agent that writes per-PDP-image-type
multimodal prompts (or per-role directives) from reference photos and PDF excerpts
before image generation.",
      "inputs": [
        "Reference product images (up to 4)",
        "image_types[]",
        "User prompt text",
        "Product spec PDF excerpt",
        "Guardrails PDF excerpt"
      ],
      "outputs": [
        "JSON prompts per image type",
        "Optional role_directives map for template fallback"
      ],
      "model_used": "Azure OpenAI gpt-5.4 (AZURE_OPENAI_PROMPT_DEPLOYMENT)",
      "tech_stack": ["Azure OpenAI Chat Completions", "FastAPI", "JSON schema
prompting"],
      "integrations": ["Azure OpenAI"],
      "origin_project": "Aira (CreAltive Studio)",
      "origin_client": "Staples",
      "vertical": "Retail / CPG",
      "status": "available",
      "typical_accuracy": null,
      "notes": "Controlled by GEMINI_PROMPT_AUTHOR env (legacy name). Falls back to
PromptGenerator templates if Azure call fails."
    },
    {
      "name": "PDP Image Generator (Gemini)",
      "version": "2.1.0",
      "category": "Sales & Revenue",
      "function_summary": "Generates Amazon-style PDP PNGs from reference images plus
authored prompts, one variant per parallel task.",
      "inputs": [
        "Reference images",
        "Per-variant prompt string",
        "provider_id gemini-flash"
      ],
      "outputs": ["PNG image bytes per variant", "base64 images in API response"],

```

```

    "model_used": "Google Gemini (default gemini-2.5-flash-image)",
    "tech_stack": ["google-genai", "FastAPI", "Pillow normalization"],
    "integrations": ["Google Gemini API"],
    "origin_project": "Aira (CreAltive Studio)",
    "origin_client": "Staples",
    "vertical": "Retail / CPG",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Default provider; supports model override via multipart model field."
  },
  {
    "name": "PDP Image Generator (Azure GPT Image)",
    "version": "2.1.0",
    "category": "Sales & Revenue",
    "function_summary": "Azure images/edits endpoint generates PDP variants from first
reference image and per-slot prompt.",
    "inputs": [
      "Reference image PNG",
      "Per-variant prompt",
      "provider_id gpt-image"
    ],
    "outputs": ["PNG per variant via Azure base64 response"],
    "model_used": "Azure OpenAI gpt-image-1.5 (AZURE_OPENAI_IMAGE_MODEL)",
    "tech_stack": ["Azure OpenAI images/edits", "requests multipart", "FastAPI"],
    "integrations": ["Azure OpenAI"],
    "origin_project": "Aira (CreAltive Studio)",
    "origin_client": "Staples",
    "vertical": "Retail / CPG",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Speed mode fastlquality via AZURE_OPENAI_IMAGE_SPEED_MODE."
  },
  {
    "name": "PDP Compliance Checker",
    "version": "2.1.0",
    "category": "Quality & Compliance",
    "function_summary": "Vision LLM scores a generated PDP image against guardrails-
only excerpts and returns pass/fail rules, warnings, suggestions, and compliance_score
0-100.",
    "inputs": [
      "Generated image file",
      "image_type",
      "Guardrails PDF text (type-focused excerpt)",
      "Optional supplementary policy notes"
    ],
    "outputs": [
      "compliance_score",
      "passed_rules",
      "failed_rules",
      "warnings",
      "suggestions"
    ],
  },

```

```

    "model_used": "Google Gemini (GEMINI_COMPLIANCE_MODEL, default gemini-2.5-
flash)",
    "tech_stack": ["google-genai", "ComplianceRulesEngine", "FastAPI"],
    "integrations": ["Google Gemini API"],
    "origin_project": "Aira (CreAltive Studio)",
    "origin_client": "Staples",
    "vertical": "Retail / CPG",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Product spec PDF required on upload but explicitly excluded from compliance
model context."
  },
  {
    "name": "Amazon PDP Video Prompt Author",
    "version": "2.1.0",
    "category": "Sales & Revenue",
    "function_summary": "Produces a full Amazon PDP video ad script (reference block,
compliance checklist, scenes, generation prompts) from product spec, guardrails, optional
user script, and reference image.",
    "inputs": [
      "Product spec PDF text",
      "Guardrails PDF text",
      "Reference image",
      "User script_or_prompt",
      "width x height x seconds"
    ],
    "outputs": ["Long video generation prompt string (Sora-clamped)"],
    "model_used": "Azure OpenAI gpt-5.4",
    "tech_stack": ["Azure OpenAI", "In-repo Amazon PDP video master template"],
    "integrations": ["Azure OpenAI"],
    "origin_project": "Aira (CreAltive Studio)",
    "origin_client": "Staples",
    "vertical": "Retail / CPG",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Human approves prompt in Editor before Sora render; can skip LLM if
approved_prompt supplied."
  },
  {
    "name": "Sora Image-to-Video Generator",
    "version": "2.1.0",
    "category": "Sales & Revenue",
    "function_summary": "Submits image-to-video jobs to Azure Sora, polls until complete,
returns MP4 for PDP video ads.",
    "inputs": [
      "Approved prompt (max 15000 UTF-8 bytes)",
      "Reference image",
      "1280x720 or 720x1280",
      "4, 8, or 12 seconds"
    ],
    "outputs": ["video/mp4 base64", "prompt_used", "meta"],
    "model_used": "Azure OpenAI Sora-2 (AZURE_OPENAI_VIDEO_DEPLOYMENT)",

```



```

    "tech_stack": ["Azure OpenAI Videos API", "Pillow image prep", "FastAPI"],
    "integrations": ["Azure OpenAI"],
    "origin_project": "Aira (CreAltive Studio)",
    "origin_client": "Staples",
    "vertical": "Retail / CPG",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Requires AZURE_OPENAI_ENDPOINT, API key, and video deployment
configured."
  }
]
}

```

Merchandising Vision Agent – Solution Summary

Client & Context

- Client: Affine Analytics (logo: acis.affineanalytics.co.in; Azure storage account `affineblog`; workspace folder `Planogram_Agentic`)
- Vertical: CPG / Retail (planogram compliance, shelf merchandising, in-store execution)
- Project Type: Agentic System (vision + LLM Q&A) with Streamlit UI, FastAPI backend, and Azure-hosted ML/data pipeline

Business Problem

The client was manually auditing retail shelf and planogram photos—counting facings by brand, reading shelf-rail prices and promotions, and spotting out-of-stock gaps—which was slow, inconsistent, and hard to scale across large image libraries in blob storage. Field teams and analysts had to visually inspect each shelf image and search folders by filename instead of describing what they needed (e.g., “chips aisle with blue bags” or “Twix price”).

Solution Overview

Merchandising Vision Agent is an end-to-end shelf-image intelligence system. Users upload or pick a shelf photo (Streamlit) or call REST APIs (`main2.py`), which triggers a Roboflow vision pipeline that detects shelf rows, crops each row, and runs YOLO product detection; outputs are cached in Azure Blob under `<image_stem>/cropped_images/`, `products_detected/`, and `metadata.json`. Users ask natural-language questions (“How many total products?”, “What is the price of Twix?”, “Which chocolate is out of stock?”); the system routes to parallel LLM analyzers on row images, then consolidates a final answer with reasoning. A semantic search sidebar embeds queries with Azure AI Vision, retrieves similar images from Azure AI Search (`image_embeddings_index`), and filters candidates with GPT vision validation before selection.

Key Features

- Image upload (Streamlit) — Upload JPG/PNG to `Input/<stem>/`, sync to Azure Blob (`planogram-images` or `vision-agent`), run vision pipeline, index into Azure AI Search

- Blob gallery browse — Paginated 4-column gallery (20/page) with SAS URLs, checkbox selection, no full page reload
- Semantic image search — Text query → Azure AI Vision vectorizeText → Azure AI Search vector query (top_k=10) → optional GPT relevance filter
- GPT image relevance validation — Parallel validation (ThreadPoolExecutor, max 10 workers); only gpt_validated images shown; gpt_reason per result
- Vision pipeline (Roboflow) — Row polygon detection → masked row crops (row_1.png...) → YOLO product boxes → shelf_summary.jpg polygon overlay
- Blob pipeline caching — run_pipeline_and_store persists crops/ annotations/metadata; is_already_processed skips re-run
- Natural-language Q&A — 🚀 Analyze runs vision fetch + llm_functions.run(); shows final answer, optional reasoning toggle
- Query routing — Keyword router (count vs generic) for numeric vs descriptive/price/promo queries
- Per-row parallel LLM analysis — count_function and generic_function run per row (up to 8 workers); price queries use full shelf + row crops
- Pipeline visuals expander — Rows summary, cropped rows, product-detection rows, latency breakdown
- Chat history expander — Legacy AutoGen groupchat.messages UI hook (when llm.manager exists)
- FastAPI: create session — POST /create-session → UUID session_id
- FastAPI: upload + pipeline — POST /upload → blob upload, Roboflow pipeline, index_single_blob_sync
- FastAPI: semantic search — POST /semantic-search → query_fetch_validate_images
- FastAPI: chat — POST /chat/{session_id} with query + image_name; persists to sessions/<id>.json
- FastAPI: chat history — GET /history/{session_id}
- FastAPI: list images — GET /images with SAS URLs
- FastAPI: cropped rows / products-detected — GET /images/{image_name}/cropped-rows, /products-detected with row indices

AI Agents & Components

Query Router (route)

- Function: Keyword-based routing to count or generic (no LLM call)
- Input: User query string
- Output: "count" or "generic"
- Model/Service: Rule-based (_COUNT_KEYWORDS, _PRICE_PROMO_KEYWORDS)
- Position in flow: Processing (before LLM functions)

Roboflow Shelf Row Detector

- Function: Detect shelf rows as polygons, crop masked rows, draw summary overlay
- Input: Full shelf image path
- Output: row_1.png... crops, shelf_summary.jpg, row paths list

- Model/Service: Roboflow workspace planogram1, project my-first-project-z5ns2, version 4, confidence 40
- Position in flow: Ingestion / processing

Roboflow Product Detector (YOLO)

- Function: Detect products per row crop; draw bounding boxes; per-class counts
- Input: Cropped row images
- Output: Annotated row_N.png, row_counts (e.g. {"object": 16})
- Model/Service: Roboflow project third-project-0rcul, version 3, confidence 40, overlap 50
- Position in flow: Processing

Azure AI Vision Embedding Service

- Function: vectorizeText / vectorizeImage for semantic search index and queries
- Input: Text prompt or image bytes
- Output: 1024-dim image_vector
- Model/Service: Azure AI Vision (model-version=2023-04-15, API 2024-02-01), endpoint image-recommendation.cognitiveservices.azure.com
- Position in flow: Ingestion (indexing) / retrieval

Azure AI Search Vector Retriever

- Function: HNSW cosine nearest-neighbor search on image_embeddings_index
- Input: Query embedding, top_k
- Output: blob_name, container, @search.score
- Model/Service: Azure AI Search (ai-search-intern.search.windows.net, index image_embeddings_index)
- Position in flow: Retrieval

GPT Image Relevance Validator

- Function: Filter vector-search candidates by visual match to query
- Input: Image bytes + user search query
- Output: {"relevant": bool, "reason": string} → gpt_validated, gpt_reason
- Model/Service: Azure OpenAI deployment gpt-5.2 (transcription-models.openai.azure.com), detail: low
- Position in flow: Validation (post-retrieval)

Shelf Count Agent (count_function)

- Function: Per-row brand recognition and product_count with structured reasoning
- Input: User query + YOLO-annotated row images
- Output: row_results[] with product_count, brand_requested, brand_detected, counting_confidence, reasoning
- Model/Service: Azure OpenAI gpt-5.2, temperature 0
- Position in flow: Processing (parallel per row)

Shelf Understanding Agent (generic_function)

- Function: Descriptive answers, price tags, promotions, stock-out cues from crops/full image

- Input: User query + cropped rows; for price/promo: original + all row crops
- Output: row_results[] with answer, reasoning
- Model/Service: Azure OpenAI gpt-5.2
- Position in flow: Processing (parallel per row)

Final Answer Consolidation Agent (final_answer_function)

- Function: Merge count_analysis + generic_analysis by task_type into one user-facing answer
- Input: Original query + intermediate JSON (task_type, both analyses)
- Output: final_answer, reasoning, user_query
- Model/Service: Azure OpenAI gpt-5.2 (text-only)
- Position in flow: Output

Legacy AutoGen Orchestrator (llm_processing.py, commented out)

- Function: Multi-agent chain: Routing_Agent → Count_Agent | Generic_Agent → Final_Answer_Agent
- Input: Shelf row images + user query
- Output: Parsed final JSON
- Model/Service: AutoGen + Azure OpenAI (superseded by llm_functions.py)

Tech Stack

- Frontend: Streamlit (main.py), custom HTML/CSS gallery (DM Sans, #4f8ef7 accent), Affine branding; CORS-enabled API consumer on ports 3000/8072
- Backend: Python, FastAPI + Uvicorn (main2.py), python-dotenv, aiohttp, ThreadPoolExecutor
- AI/ML: Azure OpenAI (gpt-5.2), Azure AI Vision (retrieval embeddings), Roboflow SDK (custom row + product models), OpenCV, Pillow
- Infrastructure: Azure Blob Storage (affineblog, containers planogram-images / vision-agent), Azure AI Search, Azure Cognitive Services
- Databases: Azure AI Search vector index (not SQL); session JSON files in sessions/
- Integrations: Roboflow API, Azure Blob/Search/Vision/OpenAI; optional key.py for local secrets

Data Flow

1. User uploads shelf image (Streamlit file uploader or POST /upload) → saved locally (Input/ or uploads/<session_id>/) and uploaded to Azure Blob
2. run_pipeline_and_store runs Roboflow row detection → row crops + shelf_summary.jpg → YOLO product annotation → uploads to <stem>/cropped_images/, products_detected/, metadata.json
3. index_single_blob_sync embeds image via Azure AI Vision → upserts doc (id, container, blob_name, image_vector) into image_embeddings_index
4. User selects image (gallery checkbox or upload) and enters a question (e.g. “How many total products?”, “price of Twix”)
5. process_image_once / fetch_processed_images loads pre-processed row images from blob or local temp_run/<run_id>/
6. llm_functions.run → route(query) → parallel count_function (annotated rows) + generic_function (cropped rows + original for price queries)

7. `final_answer_function` consolidates → `final_answer` + reasoning displayed in UI or returned via `/chat/{session_id}`
8. (Parallel path) Semantic search: text → Vision embedding → Search top-K → GPT validation → user picks validated blob → steps 4–7

External Integrations

- Azure Blob Storage: Image store, pipeline artifacts, SAS URLs for gallery/API
- Azure AI Search: Vector index `image_embeddings_index` for semantic image discovery
- Azure AI Vision: `retrieval:vectorizeText` / `vectorizeImage` embeddings
- Azure OpenAI: Chat completions for validation, counting, generic Q&A, final answer
- Roboflow: Hosted row-segmentation and product-detection models (planogram1 workspace)
- Affine Analytics branding: Logo CDN `acis.affineanalytics.co.in`

KPIs & Metrics Tracked

- **product_count**: Integer count per shelf row for requested brand (`count_function`)
- **brand_detected** / **brand_requested**: Whether target brand was found on a row
- **counting_confidence**: high | medium | low per row count
- **row_counts** (YOLO): Per-row class counts in `metadata.json` (e.g. `{"object": 16}`)
- **@search.score**: Vector similarity score from Azure AI Search
- **total_validated**: Count of GPT-approved search results (`/semantic-search`)
- **total_rows**: Number of detected shelf rows per image
- **chat_turn** / **total_turns**: Conversation turn counter per session
- Latency breakdown: Row detection + cropping, Product detection, Fetch from blob, LLM processing, Total end-to-end (seconds)
- Pipeline **status**: success, partial, or error in blob `metadata.json`

Agentic Flow (if applicable)

Upload/Select image → Roboflow row crop + YOLO annotate (cached to blob) → User question → **route(query)** → [parallel] **count_function** (annotated rows) + **generic_function** (cropped rows) → **final_answer_function** → Display **final_answer** + optional **reasoning**

Semantic search path: Text query → Vision text embedding → Azure AI Search top-10 → GPT relevance filter → User selects image → same Q&A flow above

Human-in-the-loop: Image selection (gallery checkbox), query entry, optional “Show Reasoning” toggle, Browse vs Search tabs

Notes & Caveats

- Secrets: API keys and connection strings are hardcoded as defaults in `main.py` and `key.py` (gitignored); production should use env vars only
- Legacy code: `llm_processing.py` AutoGen multi-agent flow is fully commented; active path is deterministic `llm_functions.py`

- Unused deps: requirements.txt lists pixeltable, spacy, mcp, moviepy, etc., with no imports in core app files
- No README/HTML frontend: UI is Streamlit-only; FastAPI serves API on default CORS origins localhost:3000 / 8072
- Container naming: Streamlit defaults planogram-images; key.py uses vision-agent
- Roboflow class labels: Metadata shows generic "object" class, not brand-specific YOLO labels
- GPT validation fallback: On validation error, images are included by default (relevant: true)

Agent catalog JSON

```
{
  "project": {
    "name": "Merchandising Vision Agent",
    "client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "business_problem": "The client was manually auditing retail shelf and planogram photos—counting brand facings, reading shelf-rail prices, and identifying out-of-stock gaps—which was slow and did not scale across large image libraries stored in Azure Blob.",
    "solution_summary": "An agentic shelf-vision system that runs Roboflow row and product detection on upload, caches artifacts to Azure Blob, indexes images for semantic search, and answers natural-language merchandising questions via parallel Azure OpenAI row analyzers consolidated into a final answer—with Streamlit UI and FastAPI endpoints.",
    "tech_stack": [
      "Python",
      "Streamlit",
      "FastAPI",
      "Uvicorn",
      "Azure Blob Storage",
      "Azure AI Search",
      "Azure AI Vision",
      "Azure OpenAI (gpt-5.2)",
      "Roboflow",
      "OpenCV",
      "Pillow",
      "aiohttp"
    ],
    "outcomes": "Automated shelf row segmentation, per-row product detection, semantic image discovery with GPT-filtered relevance, and NL Q&A for counts, pricing (e.g. Twix $1.45 / 99¢), and stock-out detection (e.g. Mars bars, Wispa tray empty) with reasoning and latency metrics."
  },
  "agents": [
    {
      "name": "Semantic Image Search & Validator",
      "version": "1.0",
      "category": "Sales & Revenue",
      "function_summary": "Embeds text queries with Azure AI Vision, retrieves similar shelf images from Azure AI Search, and filters results with GPT vision relevance scoring.",

```

```

    "inputs": ["Natural-language search query", "Azure Blob image corpus"],
    "outputs": ["validated_results with blob_name, sas_url, gpt_reason, @search.score"],
    "model_used": "Azure AI Vision (vectorizeText) + Azure AI Search HNSW + Azure
OpenAI gpt-5.2",
    "tech_stack": ["semantic.py", "Azure AI Search", "Azure AI Vision", "Azure OpenAI"],
    "integrations": ["Azure Blob Storage", "Azure AI Search", "Azure AI Vision", "Azure
OpenAI"],
    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "top_k=10 retrieval; GPT validation parallelized; on error includes image by
default"
  },
  {
    "name": "Roboflow Shelf Row Detector",
    "version": "4",
    "category": "Sales & Revenue",
    "function_summary": "Detects shelf rows as polygons, produces masked row crops
and a polygon-overlay summary image.",
    "inputs": ["Full shelf/planogram image"],
    "outputs": ["row_N.png crops", "shelf_summary.jpg"],
    "model_used": "Roboflow planogram1/my-first-project-z5ns2 v4",
    "tech_stack": ["detect_crop.py", "Roboflow SDK", "OpenCV"],
    "integrations": ["Roboflow API"],
    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "ROW_CONFIDENCE=40; rail_margin_ratio=0.22 for price-tag area"
  },
  {
    "name": "Roboflow Product Detector",
    "version": "3",
    "category": "Sales & Revenue",
    "function_summary": "Runs YOLO product detection on each shelf row crop and writes
annotated images with per-class counts.",
    "inputs": ["Cropped row images"],
    "outputs": ["Annotated row_N.png", "row_counts per class in metadata.json"],
    "model_used": "Roboflow planogram1/third-project-0rcul v3",
    "tech_stack": ["detect_crop.py", "Roboflow SDK", "OpenCV"],
    "integrations": ["Roboflow API"],
    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Parallel ThreadPoolExecutor up to 6 workers; class label often generic
'object'"
  },
  },

```

```

{
  "name": "Shelf Count Agent",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Per-row brand-aware product counting from YOLO-annotated shelf images with confidence and structured visual reasoning.",
  "inputs": ["User query", "Annotated row images (products_detected)"],
  "outputs": ["product_count", "brand_detected", "counting_confidence", "reasoning per row"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["llm_functions.py", "Azure OpenAI"],
  "integrations": ["Azure OpenAI"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Invoked when query matches count keywords; runs in parallel with generic agent"
},
{
  "name": "Shelf Understanding Agent",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Answers descriptive, price, promotion, and stock-out questions from cropped row images and full shelf context.",
  "inputs": ["User query", "Cropped row images", "Optional original shelf image for price/promo queries"],
  "outputs": ["answer and reasoning per row"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["llm_functions.py", "Azure OpenAI"],
  "integrations": ["Azure OpenAI"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Price/promo path uses is_price_or_promo_query() and full image plus all row crops"
},
{
  "name": "Final Answer Consolidation Agent",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Consolidates count and generic row-level analyses into a single natural-language answer based on task_type routing.",
  "inputs": ["User query", "intermediate JSON with count_analysis and generic_analysis"],
  "outputs": ["final_answer", "reasoning", "user_query"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["llm_functions.py", "Azure OpenAI"],
  "integrations": ["Azure OpenAI"],

```



```

    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Text-only call; sums product_count across rows for total queries"
  },
  {
    "name": "GPT Image Relevance Validator",
    "version": "1.0",
    "category": "Document & Data",
    "function_summary": "Strict vision-based filter that approves or rejects blob images
against a user's semantic search query.",
    "inputs": ["Image bytes", "Search query string"],
    "outputs": ["relevant boolean", "reason sentence", "gpt_validated flag"],
    "model_used": "Azure OpenAI gpt-5.2",
    "tech_stack": ["main.py", "main2.py", "semantic.py"],
    "integrations": ["Azure OpenAI", "Azure Blob Storage"],
    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Uses detail:low for cost/latency; max 8-10 parallel workers"
  }
]
}

```

Merchandising Vision Agent — Solution Summary

Client & Context

- Client: Affine Analytics (logo: acis.affineanalytics.co.in; Azure storage account `affineblog`; workspace folder `Planogram_Agentic`)
- Vertical: CPG / Retail (planogram compliance, shelf merchandising, in-store execution)
- Project Type: Agentic System (vision + LLM Q&A) with Streamlit UI, FastAPI backend, and Azure-hosted ML/data pipeline

Business Problem

The client was manually auditing retail shelf and planogram photos—counting facings by brand, reading shelf-rail prices and promotions, and spotting out-of-stock gaps—which was slow, inconsistent, and hard to scale across large image libraries in blob storage. Field teams and analysts had to visually inspect each shelf image and search folders by filename instead of describing what they needed (e.g., “chips aisle with blue bags” or “Twix price”).

Solution Overview

Merchandising Vision Agent is an end-to-end shelf-image intelligence system. Users upload or pick a shelf photo (Streamlit) or call REST APIs (`main2.py`), which triggers a Roboflow vision pipeline that detects shelf rows, crops each row, and

runs YOLO product detection; outputs are cached in Azure Blob under `<image_stem>/cropped_images/`, `products_detected/`, and `metadata.json`. Users ask natural-language questions (“How many total products?”, “What is the price of Twix?”, “Which chocolate is out of stock?”); the system routes to parallel LLM analyzers on row images, then consolidates a final answer with reasoning. A semantic search sidebar embeds queries with Azure AI Vision, retrieves similar images from Azure AI Search (`image_embeddings_index`), and filters candidates with GPT vision validation before selection.

Key Features

- Image upload (Streamlit) — Upload JPG/PNG to `Input/<stem>/`, sync to Azure Blob (`planogram-images` or `vision-agent`), run vision pipeline, index into Azure AI Search
- Blob gallery browse — Paginated 4-column gallery (20/page) with SAS URLs, checkbox selection, no full page reload
- Semantic image search — Text query → Azure AI Vision `vectorizeText` → Azure AI Search vector query (`top_k=10`) → optional GPT relevance filter
- GPT image relevance validation — Parallel validation (`ThreadPoolExecutor`, max 10 workers); only `gpt_validated` images shown; `gpt_reason` per result
- Vision pipeline (Roboflow) — Row polygon detection → masked row crops (`row_1.png...`) → YOLO product boxes → `shelf_summary.jpg` polygon overlay
- Blob pipeline caching — `run_pipeline_and_store` persists crops/annotations/metadata; `is_already_processed` skips re-run
- Natural-language Q&A —  `Analyze` runs vision fetch + `llm_functions.run()`; shows final answer, optional reasoning toggle
- Query routing — Keyword router (count vs generic) for numeric vs descriptive/price/promo queries
- Per-row parallel LLM analysis — `count_function` and `generic_function` run per row (up to 8 workers); price queries use full shelf + row crops
- Pipeline visuals expander — Rows summary, cropped rows, product-detection rows, latency breakdown
- Chat history expander — Legacy AutoGen `groupchat.messages` UI hook (when `llm.manager` exists)
- FastAPI: create session — `POST /create-session` → UUID `session_id`
- FastAPI: upload + pipeline — `POST /upload` → blob upload, Roboflow pipeline, `index_single_blob_sync`
- FastAPI: semantic search — `POST /semantic-search` → `query_fetch_validate_images`
- FastAPI: chat — `POST /chat/{session_id}` with query + `image_name`; persists to `sessions/<id>.json`
- FastAPI: chat history — `GET /history/{session_id}`
- FastAPI: list images — `GET /images` with SAS URLs
- FastAPI: cropped rows / products-detected — `GET /images/{image_name}/cropped-rows/`, `/products-detected` with row indices

AI Agents & Components

Query Router (route)

- Function: Keyword-based routing to count or generic (no LLM call)
- Input: User query string
- Output: "count" or "generic"
- Model/Service: Rule-based
(_COUNT_KEYWORDS, _PRICE_PROMO_KEYWORDS)
- Position in flow: Processing (before LLM functions)

Roboflow Shelf Row Detector

- Function: Detect shelf rows as polygons, crop masked rows, draw summary overlay
- Input: Full shelf image path
- Output: row_1.png... crops, shelf_summary.jpg, row paths list
- Model/Service: Roboflow workspace planogram1, project my-first-project-z5ns2, version 4, confidence 40
- Position in flow: Ingestion / processing

Roboflow Product Detector (YOLO)

- Function: Detect products per row crop; draw bounding boxes; per-class counts
- Input: Cropped row images
- Output: Annotated row_N.png, row_counts (e.g. {"object": 16})
- Model/Service: Roboflow project third-project-0rcul, version 3, confidence 40, overlap 50
- Position in flow: Processing

Azure AI Vision Embedding Service

- Function: vectorizeText / vectorizeImage for semantic search index and queries
- Input: Text prompt or image bytes
- Output: 1024-dim image_vector
- Model/Service: Azure AI Vision (model-version=2023-04-15, API 2024-02-01), endpoint image-recommendation.cognitiveservices.azure.com
- Position in flow: Ingestion (indexing) / retrieval

Azure AI Search Vector Retriever

- Function: HNSW cosine nearest-neighbor search on image_embeddings_index
- Input: Query embedding, top_k
- Output: blob_name, container, @search.score
- Model/Service: Azure AI Search (ai-search-intern.search.windows.net, index image_embeddings_index)
- Position in flow: Retrieval

GPT Image Relevance Validator

- Function: Filter vector-search candidates by visual match to query
- Input: Image bytes + user search query
- Output: {"relevant": bool, "reason": string} → gpt_validated, gpt_reason
- Model/Service: Azure OpenAI deployment gpt-5.2 (transcription-models.openai.azure.com), detail: low
- Position in flow: Validation (post-retrieval)

Shelf Count Agent (count_function)

- Function: Per-row brand recognition and product_count with structured reasoning
- Input: User query + YOLO-annotated row images
- Output: row_results[] with product_count, brand_requested, brand_detected, counting_confidence, reasoning
- Model/Service: Azure OpenAI gpt-5.2, temperature 0
- Position in flow: Processing (parallel per row)

Shelf Understanding Agent (generic_function)

- Function: Descriptive answers, price tags, promotions, stock-out cues from crops/full image
- Input: User query + cropped rows; for price/promo: original + all row crops
- Output: row_results[] with answer, reasoning
- Model/Service: Azure OpenAI gpt-5.2
- Position in flow: Processing (parallel per row)

Final Answer Consolidation Agent (final_answer_function)

- Function: Merge count_analysis + generic_analysis by task_type into one user-facing answer
- Input: Original query + intermediate JSON (task_type, both analyses)
- Output: final_answer, reasoning, user_query
- Model/Service: Azure OpenAI gpt-5.2 (text-only)
- Position in flow: Output

Legacy AutoGen Orchestrator (llm_processing.py, commented out)

- Function: Multi-agent chain: Routing_Agent → Count_Agent | Generic_Agent → Final_Answer_Agent
- Input: Shelf row images + user query
- Output: Parsed final JSON
- Model/Service: AutoGen + Azure OpenAI (superseded by llm_functions.py)

Tech Stack

- Frontend: Streamlit (main.py), custom HTML/CSS gallery (DM Sans, #4f8ef7 accent), Affine branding; CORS-enabled API consumer on ports 3000/8072
- Backend: Python, FastAPI + Uvicorn (main2.py), python-dotenv, aiohttp, ThreadPoolExecutor
- AI/ML: Azure OpenAI (gpt-5.2), Azure AI Vision (retrieval embeddings), Roboflow SDK (custom row + product models), OpenCV, Pillow
- Infrastructure: Azure Blob Storage (affineblog, containers planogram-images / vision-agent), Azure AI Search, Azure Cognitive Services
- Databases: Azure AI Search vector index (not SQL); session JSON files in sessions/
- Integrations: Roboflow API, Azure Blob/Search/Vision/OpenAI; optional key.py for local secrets

Data Flow

1. User uploads shelf image (Streamlit file uploader or POST /upload) → saved locally (Input/ or uploads/<session_id>/) and uploaded to Azure Blob

2. run_pipeline_and_store runs Roboflow row detection → row crops + shelf_summary.jpg → YOLO product annotation → uploads to <stem>/cropped_images/, products_detected/, metadata.json
3. index_single_blob_sync embeds image via Azure AI Vision → upserts doc (id, container, blob_name, image_vector) into image_embeddings_index
4. User selects image (gallery checkbox or upload) and enters a question (e.g. “How many total products?”, “price of Twix”)
5. process_image_once / fetch_processed_images loads pre-processed row images from blob or local temp_run/<run_id>/
6. llm_functions.run → route(query) → parallel count_function (annotated rows) + generic_function (cropped rows + original for price queries)
7. final_answer_function consolidates → final_answer + reasoning displayed in UI or returned via /chat/{session_id}
8. (Parallel path) Semantic search: text → Vision embedding → Search top-K → GPT validation → user picks validated blob → steps 4–7

External Integrations

- Azure Blob Storage: Image store, pipeline artifacts, SAS URLs for gallery/API
- Azure AI Search: Vector index image_embeddings_index for semantic image discovery
- Azure AI Vision: retrieval:vectorizeText / vectorizeImage embeddings
- Azure OpenAI: Chat completions for validation, counting, generic Q&A, final answer
- Roboflow: Hosted row-segmentation and product-detection models (planogram1 workspace)
- Affine Analytics branding: Logo CDN acis.affineanalytics.co.in

KPIs & Metrics Tracked

- **product_count**: Integer count per shelf row for requested brand (count_function)
- **brand_detected** / **brand_requested**: Whether target brand was found on a row
- **counting_confidence**: high | medium | low per row count
- **row_counts** (YOLO): Per-row class counts in metadata.json (e.g. {"object": 16})
- **@search.score**: Vector similarity score from Azure AI Search
- **total_validated**: Count of GPT-approved search results (/semantic-search)
- **total_rows**: Number of detected shelf rows per image
- **chat_turn** / **total_turns**: Conversation turn counter per session
- Latency breakdown: Row detection + cropping, Product detection, Fetch from blob, LLM processing, Total end-to-end (seconds)
- Pipeline **status**: success, partial, or error in blob metadata.json

Agentic Flow (if applicable)

Upload/Select image → Roboflow row crop + YOLO annotate (cached to blob)
 → User question → **route(query)** → [parallel] **count_function** (annotated rows)

+ **generic_function** (cropped rows) → **final_answer_function** → Display **final_answer** + optional **reasoning**
 Semantic search path: Text query → Vision text embedding → Azure AI Search top-10 → GPT relevance filter → User selects image → same Q&A flow above
 Human-in-the-loop: Image selection (gallery checkbox), query entry, optional “Show Reasoning” toggle, Browse vs Search tabs

Notes & Caveats

- Secrets: API keys and connection strings are hardcoded as defaults in main.py and key.py (gitignored); production should use env vars only
- Legacy code: llm_processing.py AutoGen multi-agent flow is fully commented; active path is deterministic llm_functions.py
- Unused deps: requirements.txt lists pixeltable, spacy, mcp, moviepy, etc., with no imports in core app files
- No README/HTML frontend: UI is Streamlit-only; FastAPI serves API on default CORS origins localhost:3000 / 8072
- Container naming: Streamlit defaults planogram-images; key.py uses vision-agent
- Roboflow class labels: Metadata shows generic "object" class, not brand-specific YOLO labels
- GPT validation fallback: On validation error, images are included by default (relevant: true)

Agent catalog JSON

```
{
  "project": {
    "name": "Merchandising Vision Agent",
    "client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "business_problem": "The client was manually auditing retail shelf and planogram photos—counting brand facings, reading shelf-rail prices, and identifying out-of-stock gaps—which was slow and did not scale across large image libraries stored in Azure Blob.",
    "solution_summary": "An agentic shelf-vision system that runs Roboflow row and product detection on upload, caches artifacts to Azure Blob, indexes images for semantic search, and answers natural-language merchandising questions via parallel Azure OpenAI row analyzers consolidated into a final answer—with Streamlit UI and FastAPI endpoints.",
    "tech_stack": [
      "Python",
      "Streamlit",
      "FastAPI",
      "Uvicorn",
      "Azure Blob Storage",
      "Azure AI Search",
      "Azure AI Vision",
      "Azure OpenAI (gpt-5.2)",
      "Roboflow",
      "OpenCV",
      "Pillow",
      "aiohttp"
    ],
  },
}
```

"outcomes": "Automated shelf row segmentation, per-row product detection, semantic image discovery with GPT-filtered relevance, and NL Q&A for counts, pricing (e.g. Twix \$1.45 / 99¢), and stock-out detection (e.g. Mars bars, Wispa tray empty) with reasoning and latency metrics."

```
},
"agents": [
{
  "name": "Semantic Image Search & Validator",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Embeds text queries with Azure AI Vision, retrieves similar shelf images from Azure AI Search, and filters results with GPT vision relevance scoring.",
  "inputs": ["Natural-language search query", "Azure Blob image corpus"],
  "outputs": ["validated_results with blob_name, sas_url, gpt_reason, @search.score"],
  "model_used": "Azure AI Vision (vectorizeText) + Azure AI Search HNSW + Azure OpenAI gpt-5.2",
  "tech_stack": ["semantic.py", "Azure AI Search", "Azure AI Vision", "Azure OpenAI"],
  "integrations": ["Azure Blob Storage", "Azure AI Search", "Azure AI Vision", "Azure OpenAI"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "top_k=10 retrieval; GPT validation parallelized; on error includes image by default"
},
{
  "name": "Roboflow Shelf Row Detector",
  "version": "4",
  "category": "Sales & Revenue",
  "function_summary": "Detects shelf rows as polygons, produces masked row crops and a polygon-overlay summary image.",
  "inputs": ["Full shelf/planogram image"],
  "outputs": ["row_N.png crops", "shelf_summary.jpg"],
  "model_used": "Roboflow planogram1/my-first-project-z5ns2 v4",
  "tech_stack": ["detect_crop.py", "Roboflow SDK", "OpenCV"],
  "integrations": ["Roboflow API"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "ROW_CONFIDENCE=40; rail_margin_ratio=0.22 for price-tag area"
},
{
  "name": "Roboflow Product Detector",
  "version": "3",
  "category": "Sales & Revenue",
  "function_summary": "Runs YOLO product detection on each shelf row crop and writes annotated images with per-class counts.",
  "inputs": ["Cropped row images"],
```

```

    "outputs": ["Annotated row_N.png", "row_counts per class in metadata.json"],
    "model_used": "Roboflow planogram1/third-project-0rcul v3",
    "tech_stack": ["detect_crop.py", "Roboflow SDK", "OpenCV"],
    "integrations": ["Roboflow API"],
    "origin_project": "Merchandising Vision Agent",
    "origin_client": "Affine Analytics",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Parallel ThreadPoolExecutor up to 6 workers; class label often generic
'object"
    },
    {
      "name": "Shelf Count Agent",
      "version": "1.0",
      "category": "Sales & Revenue",
      "function_summary": "Per-row brand-aware product counting from YOLO-annotated
shelf images with confidence and structured visual reasoning.",
      "inputs": ["User query", "Annotated row images (products_detected)"],
      "outputs": ["product_count", "brand_detected", "counting_confidence", "reasoning per
row"],
      "model_used": "Azure OpenAI gpt-5.2",
      "tech_stack": ["llm_functions.py", "Azure OpenAI"],
      "integrations": ["Azure OpenAI"],
      "origin_project": "Merchandising Vision Agent",
      "origin_client": "Affine Analytics",
      "vertical": "CPG / Retail",
      "status": "available",
      "typical_accuracy": null,
      "notes": "Invoked when query matches count keywords; runs in parallel with generic
agent"
    },
    {
      "name": "Shelf Understanding Agent",
      "version": "1.0",
      "category": "Sales & Revenue",
      "function_summary": "Answers descriptive, price, promotion, and stock-out questions
from cropped row images and full shelf context.",
      "inputs": ["User query", "Cropped row images", "Optional original shelf image for price/
promo queries"],
      "outputs": ["answer and reasoning per row"],
      "model_used": "Azure OpenAI gpt-5.2",
      "tech_stack": ["llm_functions.py", "Azure OpenAI"],
      "integrations": ["Azure OpenAI"],
      "origin_project": "Merchandising Vision Agent",
      "origin_client": "Affine Analytics",
      "vertical": "CPG / Retail",
      "status": "available",
      "typical_accuracy": null,
      "notes": "Price/promo path uses is_price_or_promo_query() and full image plus all row
crops"
    },
  },

```



```

{
  "name": "Final Answer Consolidation Agent",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Consolidates count and generic row-level analyses into a single
natural-language answer based on task_type routing.",
  "inputs": ["User query", "intermediate JSON with count_analysis and
generic_analysis"],
  "outputs": ["final_answer", "reasoning", "user_query"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["llm_functions.py", "Azure OpenAI"],
  "integrations": ["Azure OpenAI"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Text-only call; sums product_count across rows for total queries"
},
{
  "name": "GPT Image Relevance Validator",
  "version": "1.0",
  "category": "Document & Data",
  "function_summary": "Strict vision-based filter that approves or rejects blob images
against a user's semantic search query.",
  "inputs": ["Image bytes", "Search query string"],
  "outputs": ["relevant boolean", "reason sentence", "gpt_validated flag"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["main.py", "main2.py", "semantic.py"],
  "integrations": ["Azure OpenAI", "Azure Blob Storage"],
  "origin_project": "Merchandising Vision Agent",
  "origin_client": "Affine Analytics",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "Uses detail:low for cost/latency; max 8-10 parallel workers"
}
]
}

```

Mars Sales Genie (Copilot) / Unified View — Solution Summary

Client & Context

- **Client:** Mars (confectionery CPG — Snickers, Twix, Galaxy, M&M's, etc.) built by **Affine Analytics**
- **Vertical:** CPG / Retail
- **Project Type:** Agentic dashboard + data pipeline (React field/executive UI + FastAPI multi-agent copilot + shelf vision pipeline)

Business Problem

The client was manually reconciling shelf photos, planogram compliance, sales spreadsheets (Mars_Sales_Data.xlsx), and policy documents (Chocolate_Confectionery_Retail_Policy.docx, Soft_Drinks_Retail_Policy.docx) across separate tools and audits. That caused slow shelf KPI reporting, inconsistent answers to “what’s on shelf vs policy vs sales,” and field reps lacking a single prioritized route with OOS/compliance/promo context at visit time.

Solution Overview

Mars Sales Genie (Copilot) is a unified retail intelligence platform: a React 19 app (mars-sales-genie) for field executives and managers, backed by **FastAPI** (Unified View API) and an **AutoGen + Azure OpenAI** copilot (**Quin** = SQL, **Eryl** = semantic RAG, **Vision** = shelf/planogram). Shelf images upload to **Azure Blob** (planogram-images/{supermarket}/...), run **Roboflow/YOLO** row/product detection, **Azure OpenAI vision** extraction, index into **Azure AI Search** (mars-demo, image_embeddings_index), and persist KPIs to **Azure SQL** (mars.shelf_visit, mars.shelf_sku_daily, mars.Mars_Sales_Data, mars.retail_planogram_*). Users chat via **Copilot** (POST /api/unified/ask), browse **Planogram Library**, run **Shelf View** compliance/competitive analysis, simulate orders (**CatBoost** regression), and use **Global Retail Command Center** rollups.

Key Features

- **Field Executive Dashboard** (/field-executive) — GPS/demo route map, store priority cards, geofence arrival popup (15m), route variants (fastest / balanced / revenue), live KPIs from /api/kpi/*
- **Shelf View** (/dashboard?tab=shelf) — planogram grid, shelf KPI cards (shareOfShelf, avgFacings, promoCompliance, OOS/low stock/new/removed SKU counts), executive deck, image compare modals
- **Sales & Order Simulator** (/dashboard?tab=sales) — promotion/price/stock inputs → POST /api/simulator/run (CatBoost/sklearn demand + uplift/revenue/margin)
- **Competitive Intelligence** (/dashboard?tab=competitive, modal) — POST /api/competitive/analyze from vision rows → mars_visibility_score, competitor_pressure_score, placement_issues
- **Planogram Compliance Checker** (/planogram-compliance) — layout vs agreement image comparison, document rules drawer, complete report to blob
- **Planogram Library** (/planogram-library) — Azure blob browse, semantic search, unified chat with agent trace, upload pipeline
- **Global Retail Command Center** (/global-command-center) — filtered store/product rollups, OOS/refill/transit/expiry KPIs, SQL-backed /api/global-retail/overview
- **Embedded Copilot** — CopilotModal → unifiedAsk() with optional shelf blob attachment and bar-chart visualizations
- **Store sidebar & region selector** — store queue scoped to supermarket blob prefix (e.g. More_Supermarket)

- **Route Planner / Stores / Orders** — demo pages with mock src/data/
* when backend offline
- **Rep Performance modal** — rep stats from mock repPerformance.ts
- **Notifications** — alert feed with read/unread
- **Login gate** — isMarsCopilotAuthed() session gate on protected routes
- **Streamlit legacy UI** (backend/streamlit_app.py) — alternate unified + upload console
- **Document/structured upload pipeline** — index PDF/DOCX to Azure Search; ingest CSV/XLSX to SQL for Quin
- **Shelf compliance cache & sidecars** — blob-backed complete-report, agreement-compliance-result, compliance-sidecars
- **Field executive visit persistence** — fieldExecutiveVisitStorage merges store + shelf recommendations
- **Executive action cards** — /api/promo-actions, /api/recommended-actions, /api/compliance-actions with optional LLM title/subtitle enrichment

AI Agents & Components

Pipeline Intent Classifier (unified_intent.classify_pipeline_mode)

- **Function:** Routes each user question to vision_only, vision_then_unified, or unified_only; detects multi-shelf comparisons for SQL-only path
- **Input:** question, has_image
- **Output:** JSON { mode, run_vision_pipeline, reason, shelf_comparison_sql_route }
- **Model/Service:** Azure OpenAI deployment gpt-5.2 (GPT4_LLM_MODEL_DEPLOYMENT_NAME)
- **Position in flow:** Ingestion / orchestration (before Quin/Eryl/Vision)

Unified Routing Agent (routing_agent in Unified_Mdb.py)

- **Function:** Decides SQL vs semantic vs vision and rewrites sql_question / semantic_question / vision_question
- **Input:** User question JSON, optional [Vision analysis ...] block, SQL_Meta_Data schema introspection
- **Output:** Strict JSON route plan (route, needs_sql, needs_semantic, needs_vision, primary_flow, etc.)
- **Model/Service:** Azure OpenAI via AutoGen llm_config → gpt-5.2
- **Position in flow:** Orchestration (GroupChat entry)

Quin — SQL Generator / Executor / Critic chain

- **Function:** Generate SQL Server queries on mars.* tables, execute, critique, emit structured SQL answers (stock, orders, Mars_Sales_Data revenue, shelf_visit metrics)
- **Input:** sql_question, table metadata from metadata.get_schema_tables
- **Output:** Query results + Insight_Generator narrative; analysis_type SQL-based
- **Model/Service:** Azure OpenAI gpt-5.2 + AutoGen agents: Sql_Generator, Sql_Executor, Sql_tool, Sql_Execution_Critic, Insight_Generator
- **Position in flow:** Processing / validation

Eryl — Semantic RAG chain

- **Function:** Vector + semantic search over indexed policies/docs; query_transformer, Selector_agent, retriever, llm_answer_maker, critic_agent
- **Input:** semantic_question, hints from Vector_Data (policy, FIFO, planogram, allergen, etc.)
- **Output:** Retrieval-grounded answer from index mars-demo on ai-search-intern.search.windows.net
- **Model/Service:** Azure OpenAI + Azure AI Search (text-embedding-3-small embeddings)
- **Position in flow:** Processing (document/policy questions)

Final Answer Rewriter (_rewrite_final_answer_with_llm)

- **Function:** De-duplicates and structures combined vision+SQL+semantic draft for end users
- **Input:** Original question + draft answer
- **Output:** { final_answer_text } JSON
- **Model/Service:** Azure OpenAI gpt-5.2, temperature 0
- **Position in flow:** Output

Planogram Vision — Shelf Levels (shelf_levels_function)

- **Function:** Full-shelf vision JSON by band (eye_level, middle_shelf, bottom_shelf) with product_name, price, shelf_share_percent, stock_status, misaligned
- **Input:** Single full shelf image (base64)
- **Output:** shelf_levels.json structure
- **Model/Service:** Azure OpenAI vision (planogram_agentic/key.py deployment gpt-5.2)
- **Position in flow:** Ingestion (upload pipeline)

Planogram Vision — Count Agent (count_function)

- **Function:** Per-row brand/product counts from YOLO-annotated row images
- **Input:** Annotated row images + user query (brand-specific count)
- **Output:** { counts, reasoning }
- **Model/Service:** Azure OpenAI vision gpt-5.2
- **Position in flow:** Processing (keyword route "count")

Planogram Vision — Generic Agent (generic_function)

- **Function:** Descriptive shelf-row understanding (non-count queries)
- **Input:** Cropped masked row images
- **Output:** Structured JSON per generic prompts
- **Model/Service:** Azure OpenAI vision gpt-5.2
- **Position in flow:** Processing (route "generic")

Planogram Vision — Final Answer (final_answer_function)

- **Function:** Text-only synthesis after row-level analysis
- **Input:** Prior vision JSON + user question
- **Output:** Natural-language final shelf answer
- **Model/Service:** Azure OpenAI gpt-5.2

- **Position in flow:** Output

Planogram Vision — Daily KPI Snapshot (daily_kpi_function)

- **Function:** Emits SQL-ready daily KPI JSON at upload time
- **Input:** Full shelf image (+ optional shelf_levels hint)
- **Output:** KPI dict normalized into mars.shelf_visit / mars.shelf_sku_daily
- **Model/Service:** Azure OpenAI vision gpt-5.2
- **Position in flow:** Ingestion → SQL persistence

Shelf & Layout Compliance Vision (shelf_compliance_llm.py)

- **Function:** Compares actual shelf vs layout/agreement images; emits shelf_compliance_rows, issues, clause verdicts; scores via compliance_scoring.py
- **Input:** Actual + reference images, agreement document text, layout facts (void_count, misplacement_count)
- **Output:** ComplianceAnalysisResult with placement_score, facings_score, pricing_score, layout_match_score, document_rule_score, recommended actions
- **Model/Service:** Azure OpenAI vision gpt-5.2 + deterministic scoring
- **Position in flow:** Validation (shelf-compliance API)

Competitive Intelligence Scorer (competitive_scoring.py)

- **Function:** Deterministic Mars vs competitor shelf scores from vision rows (not LLM for numeric scores)
- **Input:** Vision-extracted brand rows with shelf levels and facings
- **Output:** mars_visibility_score, competitor_pressure_score, overall_competitive_score, brand_occupancy, placement_issues
- **Model/Service:** Rule-based math (weights: eye_level 1.0, middle 0.75, bottom 0.45)
- **Position in flow:** Output (post-vision)

Executive Action Text Generator (llm_action_text_service.py)

- **Function:** Short Mars retail action card titles/subtitles for promo/compliance/recommended actions
- **Input:** Action dict (store, flags, KPI context)
- **Output:** Enriched { title, subtitle } (max 8 / 22 words)
- **Model/Service:** Azure OpenAI deployment from planogram_agentic/key.py
- **Position in flow:** Output (field executive APIs)

Store Task Text Generator (llm_task_text_service.py)

- **Function:** Enriches store visit tasks with LLM-generated “why store” copy
- **Input:** Store tasks payload from SQL
- **Output:** Enriched task descriptions
- **Model/Service:** Azure OpenAI
- **Position in flow:** Processing (/api/store-tasks)

Roboflow / YOLO Row & Product Detection (detect_crop.py)

- **Function:** Detects shelf rows and products; writes products_detected, cropped rows, metadata.json to blob
- **Input:** Shelf image bytes

- **Output:** Annotated crops under {stem}/... in Azure Blob
- **Model/Service:** Roboflow API (planogram1 workspace, row + product projects)
- **Position in flow:** Ingestion (before vision LLMs)

Sales Simulator Regressor (sales_simulator_regression.py)

- **Function:** Predicts weekly units/revenue/margin under promo/price/stock caps
- **Input:** promotion_discount, price, sku_id (snickers_50g, mars_bar, twix_46g), store_id, horizon_weeks, stock_units
- **Output:** uplift, realized sales, risk flags, order suggestions
- **Model/Service:** CatBoostRegressor (fallback sklearn pipeline) trained on Soft_Drinks_Sales_Q1_2026.xlsx
- **Position in flow:** Processing (/api/simulator/run)

Image Semantic Search Validator (planogram_agentic/semantic.py)

- **Function:** Azure AI Vision embeddings + vector search + GPT validation of matches
- **Input:** Natural-language query, supermarket prefix filter
- **Output:** Ranked blob images with validation reasons
- **Model/Service:** Azure AI Vision + Azure AI Search image_embeddings_index + Azure OpenAI
- **Position in flow:** Retrieval (Planogram Library search)

Tech Stack

- **Frontend:** React 19, TypeScript, Vite 8, Tailwind CSS v4, React Router 7, Recharts 2.x, react-markdown, remark-gfm
- **Backend:** Python 3, FastAPI, Uvicorn, Streamlit (legacy), SQLAlchemy 2, pyodbc, pandas, openpyxl, python-docx, PyPDF2
- **AI/ML:** Azure OpenAI (gpt-5.2, gpt-4o-08-06 GraphRAG config, text-embedding-3-small), AutoGen/pyautogen GroupChat, Roboflow, OpenCV, Pillow, CatBoost, scikit-learn, tiktoken, spaCy (requirements)
- **Infrastructure:** Azure Blob Storage (affineblog / planogram-images), Azure AI Search, Azure AI Vision, Azure SQL (sqlinternserver.database.windows.net / sqlintern_database), optional VM deploy (ports.config.json host 20.120.212.78)
- **Databases:** Azure SQL Server
schema mars — shelf_visit, shelf_sku_daily, Mars_Sales_Data, retail_planogram_stocks, retail_planogram_sales, retail_planogram_store; commented SAP HANA (hdbcli) and Salesforce (simple_salesforce) imports in metadata.py
- **Integrations:** Azure OpenAI, Azure Blob, Azure AI Search, Azure AI Vision, Roboflow, Microsoft SQL Server, Salesforce SDK (wired, not primary path), FFmpeg/librosa/moviepy (media deps in requirements), MCP package listed

Data Flow

1. Field rep or analyst captures/selects a **shelf image** (or policy/sales file) in React UI or Streamlit.

2. **Upload** (POST /api/planogram/upload or unified ask with file) writes {supermarket}/{filename} to **Azure Blob**.
3. **Roboflow/YOLO** (detect_and_crop_rows, annotate_rows_with_yolo) produces row crops, product detections, metadata.json under {stem}/.
4. **Azure OpenAI vision** functions extract shelf levels, row counts, daily KPI JSON → normalized into **mars.shelf_visit** and **mars.shelf_sku_daily** via mars_shelf_daily.py.
5. Image is **indexed** in Azure AI Search (semantic.py / upload_pipeline.py) for later semantic image search.
6. User asks a **natural-language question** in Copilot or Planogram Library → **classify_pipeline_mode** chooses vision/SQL/semantic path.
7. If vision: **planogram_bridge.run_planogram_vision** injects [Vision analysis ...] into the unified question.
8. **AutoGen GroupChat** runs **routing_agent** → **Quin SQL chain and/or Eryl RAG chain** against SQL metadata + mars-demo search index.
9. **Final Answer Rewriter** cleans the combined response; API returns response + agent_trace.
10. **Shelf compliance** path compares actual vs layout/agreement → vision rows → **deterministic KPI scores** → recommended actions saved to blob/cache.
11. **Competitive analyze** consumes vision rows → **competitive_scoring** → dashboard executive metrics.
12. **Field Executive** pulls live /api/kpi/critical-stores, oos-alerts, executive actions, store tasks → map prioritization and visit persistence.
13. **Sales simulator** reads promo/price inputs → **CatBoost** prediction → order/uplift UI on Dashboard sales tab.
14. **Global Command Center** aggregates filtered mock+SQL overview for enterprise rollups (stores, OOS, refill, sales velocity).

External Integrations

- **Azure Blob Storage (planogram-images):** Canonical shelf image + pipeline artifact store (crops, JSON, reports)
- **Azure AI Search (mars-demo, image_embeddings_index):** Document RAG for Eryl + image vector search
- **Azure OpenAI (transcription-models.openai.azure.com, deployment gpt-5.2):** All LLM/vision/chat/embeddings
- **Azure AI Vision:** Image embedding generation for semantic shelf search
- **Azure SQL Server:** Transactional shelf KPIs, Mars sales, retail planogram stock/sales/store tables
- **Roboflow:** YOLO row segmentation and product detection models
- **Microsoft SQL via ODBC Driver 18:** Database connectivity
- **Salesforce / SAP HANA:** Referenced in metadata.py imports (secondary/commented; not primary active path in current flows)

KPIs & Metrics Tracked

- **compliance_percent / complianceScore:** Overall shelf/planogram compliance (visit + compliance panel)

- **shareOfShelf / shelf_share_percent:** Mars SKU horizontal facings share per shelf band
- **avgFacings / facings_score:** Actual vs expected facings
- **promoCompliance / promo_compliant / promo_score:** Promotion adherence on shelf
- **outOfStockCount / stock_status / OOS:** Out-of-stock and OOS Risk flags (/api/kpi/oos-alerts)
- **lowStockCount / low_stock:** Low stock facings
- **newSkuCount / new_sku / removedSkuCount / removed_sku:** SKU change detection vs prior visit
- **placement_score / placement_quality:** Product slot adherence
- **pricing_score:** Shelf label vs expected pricing
- **layout_match_score / sequence_score / void_penalty_score:** Layout sequence + void/misplacement penalties
- **document_rule_score:** Weighted clause pass/fail from retail policy agreement text
- **mars_visibility_score / competitor_pressure_score / overall_competitive_score:** Competitive intelligence deck
- **shelf_position_score / brand_occupancy.percentage:** Weighted eye/middle/bottom placement strength
- **blur_score, glare_pct, occlusion_pct, coverage_pct, quality_confidence:** Image quality on upload
- **void_count, misplacement_count:** Layout gap metrics
- **velocity / promoUplift / storeSalesInr / ordersTransactions / avgBasketValue / unitsSold:** Dashboard commercial KPI trends (mock series in kpis.ts, SQL-backed where wired)
- **routeEfficiency / productiveCalls / drive time / high-risk stops / revenue uplift index:** Route planner optimization compare
- **Global: totalStoresCovered, totalMarsShelvesAssigned, fillRatePercent, onShelfAvailabilityPercent, globalSalesPerformanceIndex, expiryRiskUnits, productsInTransit, storesRequiringRefill:** Global Retail Command Center
- **sellThroughIndex / stockCoverDays / sales7dUnits / sales28dUnits:** Per-store and per-product rollups
- **critical store flags:** OOS Risk, Low Sales, Compliance Delayed (CRITICAL_STORES_KPI_SQL)
- **simulator outputs:** projected units, revenue uplift, margin, stock-constrained realized sales

Agentic Flow (if applicable)

Trigger: User submits question (+ optional image/blob) via POST /api/unified/ask, Copilot modal, Planogram Library chat, or Streamlit.

Flow: User question → **Intent Classifier** (vision_only | vision_then_unified | unified_only; multi-shelf → SQL-only override) → **[Vision branch]** Roboflow crop → **Count/Generic/Final vision LLMs** → inject [Vision analysis] → **AutoGen GroupChat** → **routing_agent** decides SQL/semantic/vision mix

→ **[Quin]** Sql_Generator → Sql_Executor → Sql_Execution_Critic →
Insight_Generator **OR** **[Eryl]** query_transformer → Selector → retriever →
llm_answer_maker → critic_agent → **Final Answer Rewriter** →
JSON response + agent_trace → UI markdown/charts.

Human-in-the-loop: Store/supermarket selection in sidebar, blob picker in Library, manual compliance report completion, “Mark shelf fixed” modal, route variant choice, login gate.

Parallel paths: Upload-time pipeline skips chat (blob → Roboflow → vision KPIs → SQL); shelf-compliance and competitive endpoints run dedicated vision+scoring without full GroupChat.

Notes & Caveats

- frontend/README.md describes “mock data only,” but the app **does** call live APIs (unifiedApi.ts, planogramApi.ts, fieldExecutiveApi.ts, competitiveApi.ts) when the FastAPI backend is running; many demo pages still use src/data/* fallbacks.
- **Secrets are hard-coded** in backend/API_unified.py, backend/planogram_agentic/key.py, and metadata.py (storage keys, OpenAI keys, SQL password) — must move to env/Key Vault before production or sharing.
- Run backend: uvicorn backend.api.main:app (ports 8002/8765 per config); frontend proxies /api via Vite (frontendPort 8003).
- Repo also ships Mars-Retail-Vision_Usecase-and-Code-Summary.docx and generate_usecase_report_docx.py for formal documentation.
- GraphRAG config references secondary endpoint aipractices-east2.openai.azure.com with gpt-4o-08-06 — may be alternate/legacy embedding path.

Agent catalog JSON

```
{
  "project": {
    "name": "Mars Sales Genie (Copilot) / Unified View",
    "client": "Mars",
    "vertical": "CPG / Retail",
    "business_problem": "Field and HQ teams manually reconciled shelf photos, planogram compliance, Mars sales spreadsheets, and retail policy documents across disconnected tools, causing slow KPI reporting and inconsistent answers about on-shelf execution versus policy and sales data.",
    "solution_summary": "Unified retail copilot and dashboard: React field-executive UI plus FastAPI backend that uploads shelf images to Azure Blob, runs Roboflow/YOLO and Azure OpenAI vision, persists KPIs to Azure SQL, indexes content in Azure AI Search, and answers questions via AutoGen-orchestrated Quin (SQL) and Eryl (semantic RAG) agents with optional vision enrichment and compliance/competitive scoring.",
    "tech_stack": [
      "React 19",
      "TypeScript",
      "Vite 8",
```

```

    "Tailwind CSS v4",
    "FastAPI",
    "Uvicorn",
    "Azure OpenAI gpt-5.2",
    "Microsoft AutoGen",
    "Azure Blob Storage",
    "Azure AI Search",
    "Azure AI Vision",
    "Azure SQL Server",
    "Roboflow YOLO",
    "CatBoost",
    "SQLAlchemy",
    "pyodbc",
    "Recharts"
  ],
  "outcomes": "Single interface for shelf vision KPIs, policy-aware Q&A, sales SQL analytics, route prioritization by OOS/compliance risk, planogram semantic search, and promo/order simulation—reducing manual audit reconciliation time and standardizing Mars field execution insights."
},
"agents": [
  {
    "name": "Pipeline Intent Classifier",
    "version": "1.0",
    "category": "Document & Data",
    "function_summary": "Classifies each user question into vision_only, vision_then_unified, or unified_only and flags multi-shelf comparisons for SQL-only routing.",
    "inputs": ["question", "has_image"],
    "outputs": ["mode", "run_vision_pipeline", "reason", "shelf_comparison_sql_route"],
    "model_used": "Azure OpenAI gpt-5.2",
    "tech_stack": ["Azure OpenAI", "Python"],
    "integrations": ["Azure OpenAI"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Implemented in backend/unified_intent.py; runs before planogram_bridge and Unified_Mdb."
  },
  {
    "name": "Quin SQL Agent Chain",
    "version": "1.0",
    "category": "Sales & Revenue",
    "function_summary": "AutoGen multi-agent chain that generates, executes, and critiques SQL Server queries on mars schema tables (Mars_Sales_Data, shelf_visit, retail_planogram_stocks, etc.) and returns structured sales/inventory insights.",
    "inputs": ["sql_question", "SQL_Meta_Data schema", "optional vision_context block"],
    "outputs": ["sql_query results", "SQL-based narrative answer", "analysis_type SQL-based"],
    "model_used": "Azure OpenAI gpt-5.2 via AutoGen",

```

```

    "tech_stack": ["AutoGen", "Azure OpenAI", "SQLAlchemy", "pyodbc", "Azure SQL"],
    "integrations": ["Azure OpenAI", "Azure SQL Server"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Agents: routing_agent, Sql_Generator, Sql_Executor, Sql_tool,
Sql_Execution_Critic, Insight_Generator in Unified_Mdb.py."
  },
  {
    "name": "Eryl Semantic RAG Agent Chain",
    "version": "1.0",
    "category": "Document & Data",
    "function_summary": "Retrieves and answers from indexed retail policy and
unstructured documents (Chocolate_Confectionery_Retail_Policy.docx, emails, guidelines)
using Azure AI Search vector + semantic retrieval.",
    "inputs": ["semantic_question", "Vector_Data routing hints"],
    "outputs": ["retrieval-grounded answer", "Semantic-based analysis_type"],
    "model_used": "Azure OpenAI gpt-5.2 + text-embedding-3-small",
    "tech_stack": ["AutoGen", "Azure OpenAI", "azure-search-documents"],
    "integrations": ["Azure OpenAI", "Azure AI Search index mars-demo"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Agents: query_transformer, Selector_agent, retriever, llm_answer_maker,
critic_agent."
  },
  {
    "name": "Planogram Vision LLM Suite",
    "version": "1.0",
    "category": "Quality & Compliance",
    "function_summary": "Azure OpenAI vision functions for shelf-level product extraction,
row counting, generic row description, daily KPI JSON, and final natural-language shelf
answers after Roboflow cropping.",
    "inputs": ["full shelf image", "YOLO-annotated row crops", "masked row images", "user
query"],
    "outputs": ["shelf_levels.json", "counts JSON", "daily KPI snapshot", "final shelf answer
text"],
    "model_used": "Azure OpenAI gpt-5.2 vision",
    "tech_stack": ["Azure OpenAI", "OpenCV", "Pillow", "Roboflow"],
    "integrations": ["Azure OpenAI", "Azure Blob Storage", "Roboflow"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "backend/planogram_agentic/llm_functions.py — count_function,
generic_function, final_answer_function, shelf_levels, daily_kpi."
  },

```

```

{
  "name": "Shelf Layout Compliance Vision Agent",
  "version": "1.0",
  "category": "Quality & Compliance",
  "function_summary": "Compares actual shelf photos against layout and agreement
references; emits compliance rows and issues; deterministic scoring for placement,
facings, pricing, layout match, and document clauses.",
  "inputs": ["actual shelf image", "layout reference image", "agreement document text",
"void_count", "misplacement_count"],
  "outputs": ["shelf_compliance_rows", "issues list", "placement_score", "facings_score",
"pricing_score", "layout_match_score", "document_rule_score", "recommended_actions"],
  "model_used": "Azure OpenAI gpt-5.2 vision + deterministic compliance_scoring.py",
  "tech_stack": ["Azure OpenAI", "Python"],
  "integrations": ["Azure OpenAI", "Azure Blob Storage"],
  "origin_project": "Mars Sales Genie (Copilot) / Unified View",
  "origin_client": "Mars",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "POST /api/planogram/shelf-compliance and related complete-report
endpoints."
},
{
  "name": "Competitive Shelf Intelligence Scorer",
  "version": "1.0",
  "category": "Sales & Revenue",
  "function_summary": "Computes Mars visibility, competitor pressure, promo and shelf
position scores from vision-extracted brand facings using weighted shelf-level rules.",
  "inputs": ["vision shelf rows with brand, facings, shelf_level"],
  "outputs": ["mars_visibility_score", "competitor_pressure_score",
"overall_competitive_score", "brand_occupancy", "placement_issues"],
  "model_used": "Deterministic (no LLM for scores)",
  "tech_stack": ["Python"],
  "integrations": [],
  "origin_project": "Mars Sales Genie (Copilot) / Unified View",
  "origin_client": "Mars",
  "vertical": "CPG / Retail",
  "status": "available",
  "typical_accuracy": null,
  "notes": "backend/competitive_scoring.py; POST /api/competitive/analyze."
},
{
  "name": "Final Answer Rewriter",
  "version": "1.0",
  "category": "Document & Data",
  "function_summary": "Post-processes combined vision+SQL+semantic draft answers
into concise structured user-facing text without adding new facts.",
  "inputs": ["original question", "draft answer"],
  "outputs": ["final_answer_text"],
  "model_used": "Azure OpenAI gpt-5.2",
  "tech_stack": ["Azure OpenAI"],
  "integrations": ["Azure OpenAI"],

```

```

    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Toggle via UNIFIED_ENABLE_FINAL_REWRITE env in Unified_Mdb.py."
  },
  {
    "name": "Roboflow Shelf Row Detector",
    "version": "1.0",
    "category": "Supply Chain & Logistics",
    "function_summary": "Segments shelf rows and detects product bounding boxes;
writes cropped and annotated artifacts to Azure Blob for downstream vision LLMs.",
    "inputs": ["shelf image bytes"],
    "outputs": ["row crops", "products_detected annotations", "metadata.json"],
    "model_used": "Roboflow YOLO (row project v4, product project v3)",
    "tech_stack": ["Roboflow", "OpenCV"],
    "integrations": ["Roboflow API", "Azure Blob Storage"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "planogram_agentic/detect_crop.py; triggered on upload via
planogram_upload.py."
  },
  {
    "name": "Mars Sales Order Simulator",
    "version": "1.0",
    "category": "Sales & Revenue",
    "function_summary": "CatBoost/sklearn regression predicts demand and realized sales
under promotion, price, and stock constraints for Snickers/Mars/Twix SKUs.",
    "inputs": ["promotion_discount", "price", "sku_id", "store_id", "horizon_weeks",
"stock_units"],
    "outputs": ["projected units", "revenue uplift", "margin", "order recommendation", "risk
flags"],
    "model_used": "CatBoostRegressor (fallback sklearn)",
    "tech_stack": ["CatBoost", "scikit-learn", "pandas"],
    "integrations": [],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "Trained from Soft_Drinks_Sales_Q1_2026.xlsx; POST /api/simulator/run."
  },
  {
    "name": "Executive Action Card Writer",
    "version": "1.0",
    "category": "Sales & Revenue",
    "function_summary": "Generates short action-oriented title and subtitle text for field
executive promo, compliance, and recommended action cards."
  }

```

```

    "inputs": ["action dict with store and KPI flags"],
    "outputs": ["title", "subtitle"],
    "model_used": "Azure OpenAI (planogram_agentic deployment)",
    "tech_stack": ["Azure OpenAI"],
    "integrations": ["Azure OpenAI"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "app/services/llm_action_text_service.py; enrich_actions_with_llm."
  },
  {
    "name": "Planogram Image Semantic Search",
    "version": "1.0",
    "category": "Document & Data",
    "function_summary": "Embeds shelf images with Azure AI Vision, vector-searches
Azure AI Search by supermarket, and validates matches with GPT before returning blob
hits.",
    "inputs": ["natural language query", "supermarket prefix"],
    "outputs": ["ranked blob images", "validation reasons", "SAS URLs"],
    "model_used": "Azure AI Vision embeddings + Azure OpenAI gpt-5.2 validation",
    "tech_stack": ["Azure AI Vision", "Azure AI Search", "Azure OpenAI"],
    "integrations": ["Azure AI Vision", "Azure AI Search", "Azure Blob Storage", "Azure
OpenAI"],
    "origin_project": "Mars Sales Genie (Copilot) / Unified View",
    "origin_client": "Mars",
    "vertical": "CPG / Retail",
    "status": "available",
    "typical_accuracy": null,
    "notes": "POST /api/planogram/semantic-search; index image_embeddings_index."
  }
]
}

```